

# Deep Learning for Computer Vision

114-1/Fall 2025; 9:30am @ BL113

<https://cool.ntu.edu.tw/courses/53063> (NTU COOL)

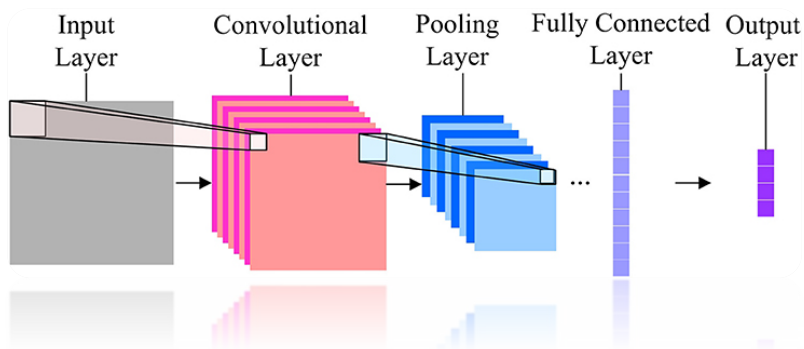
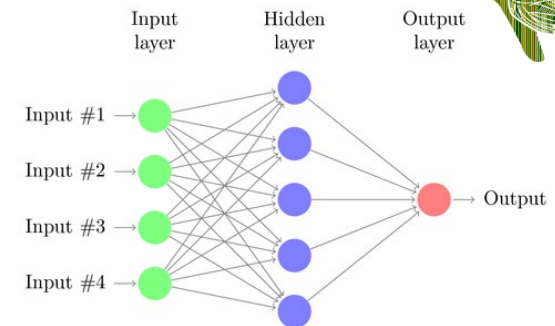
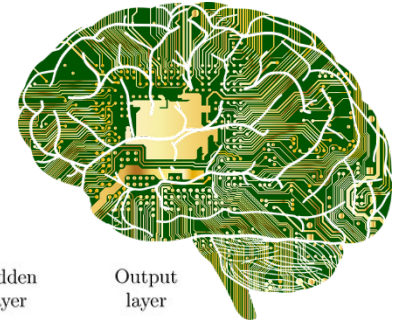
<http://vllab.ee.ntu.edu.tw/dlcv.html> (Public website)

Yu-Chiang Frank Wang 王鈺強, Professor

Dept. Electrical Engineering, National Taiwan University

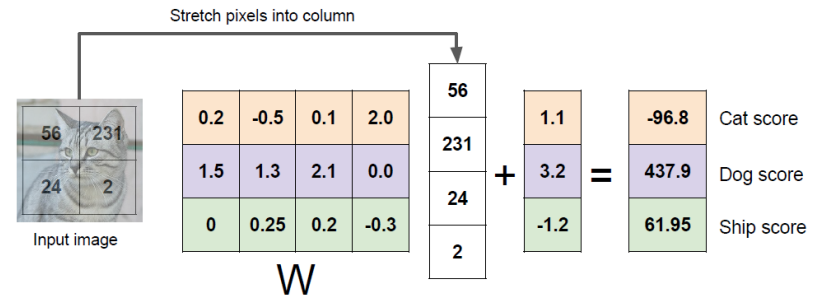
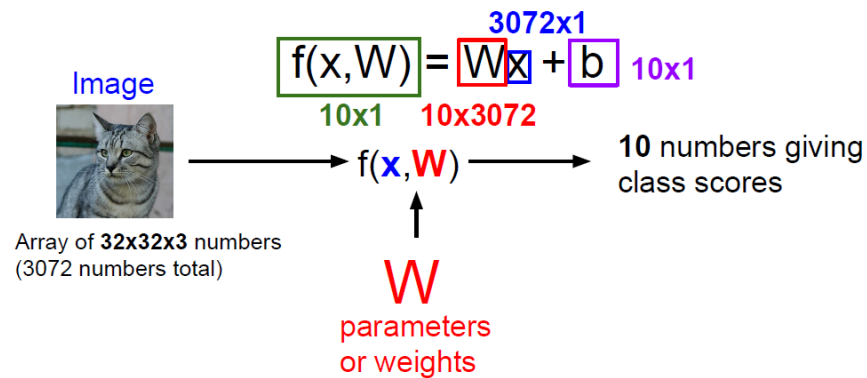
# What to Cover Today...

- Convolution Neural Networks (CNN)
  - Design of CNN
  - Variants of CNNs
  - Training Techniques for CNN
  - Self-Supervised Learning for CNN

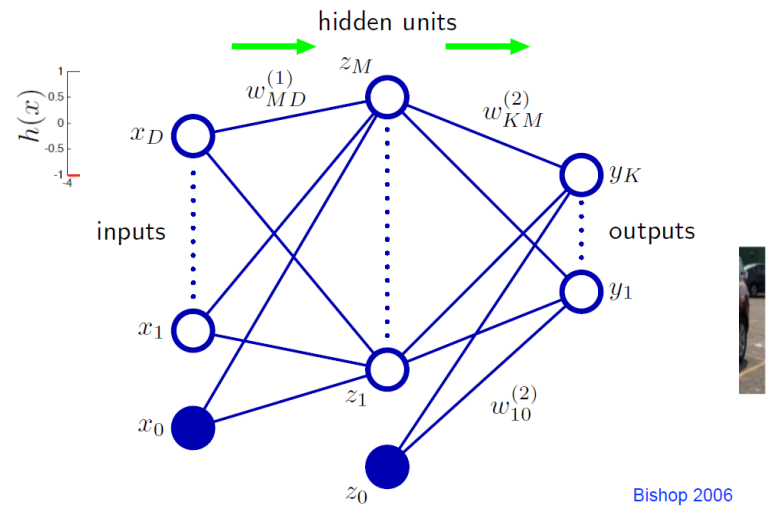
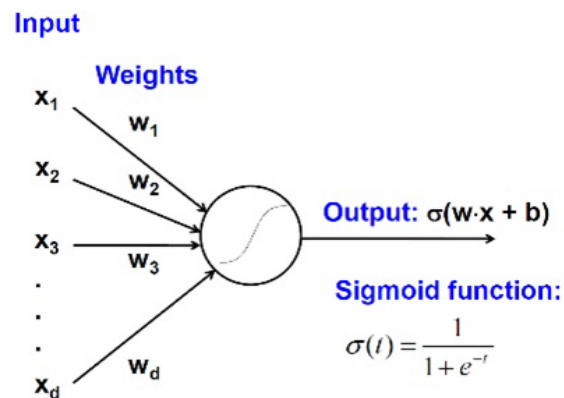


# Recap: From Linear Classifiers to Neural Nets

- Linear Classifier



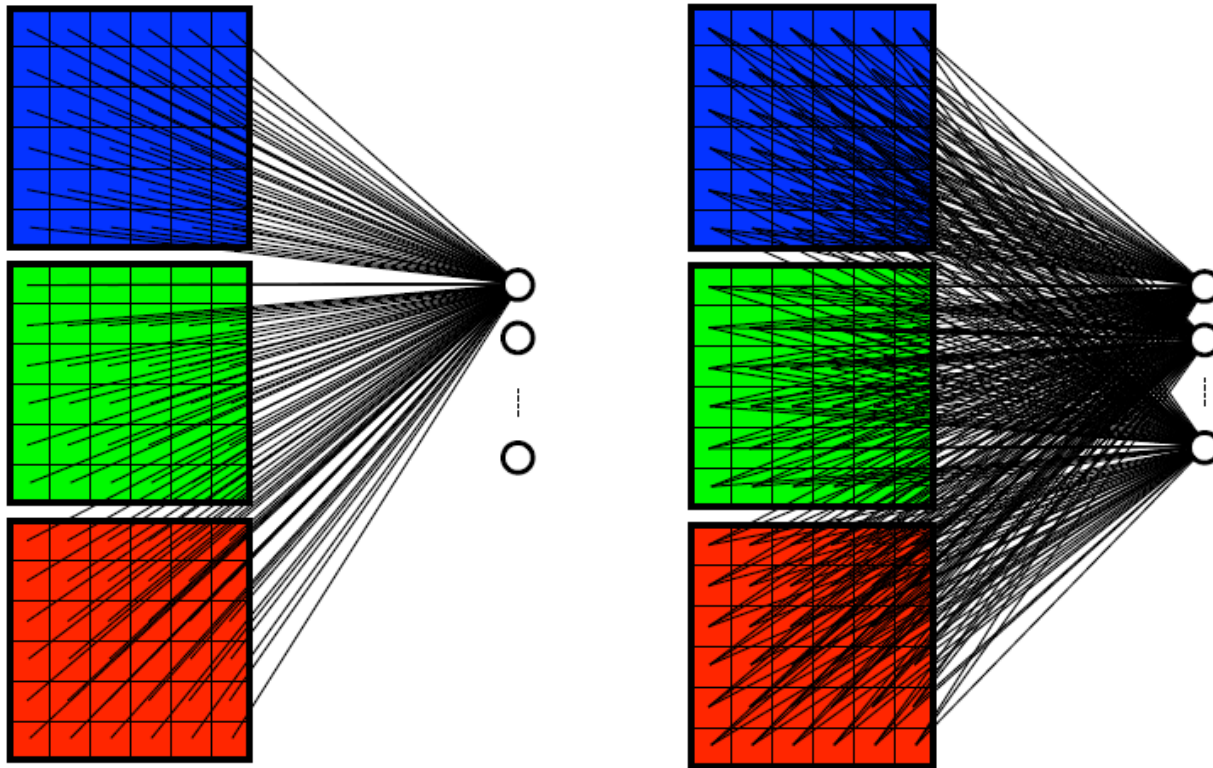
- Neural Network (Multilayer Perceptron)



Bishop 2006

# Convolutional Neural Networks

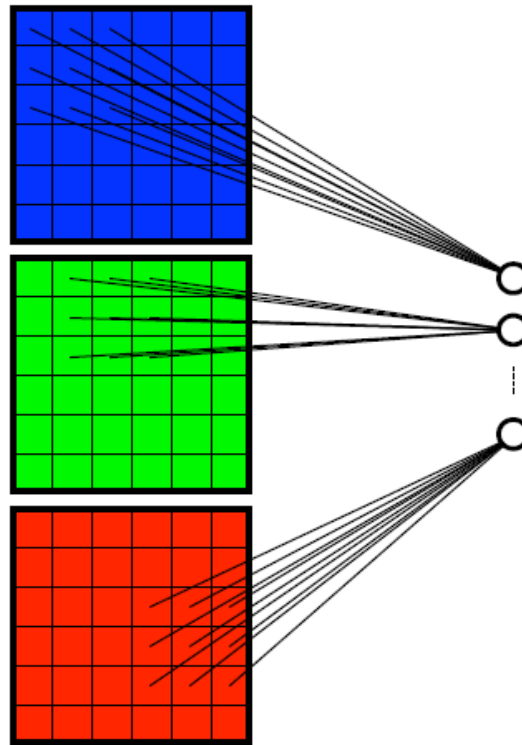
- How many weights for MLPs for images?





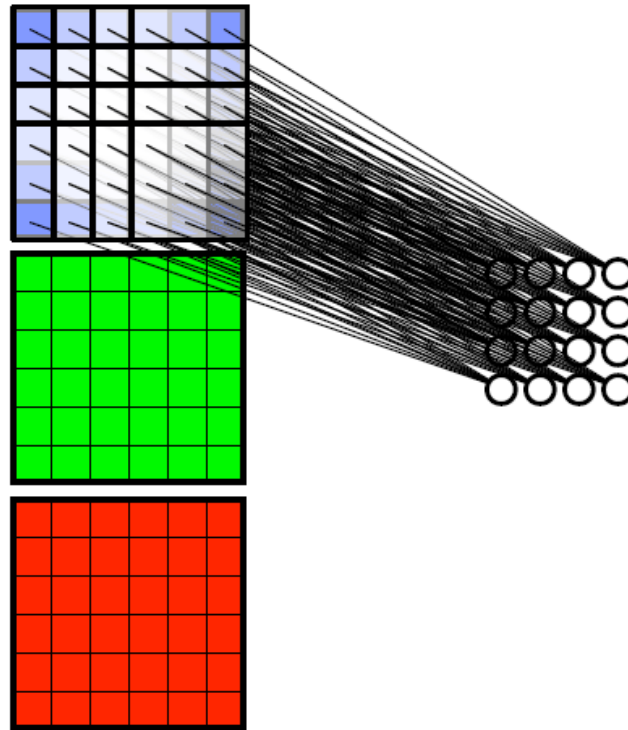
# Convolutional Neural Networks

- Property I of CNN: Local Connectivity
  - Each neuron takes info only from a **neighborhood** of pixels.

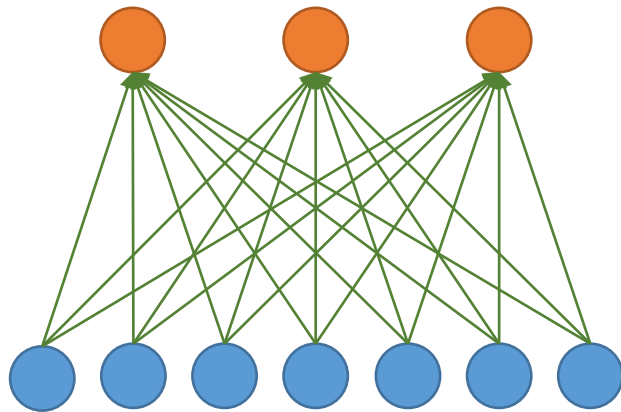


# Convolutional Neural Networks

- Property II of CNN: Weight Sharing
  - Neurons connecting each pixel and its neighborhoods have **identical** weights.



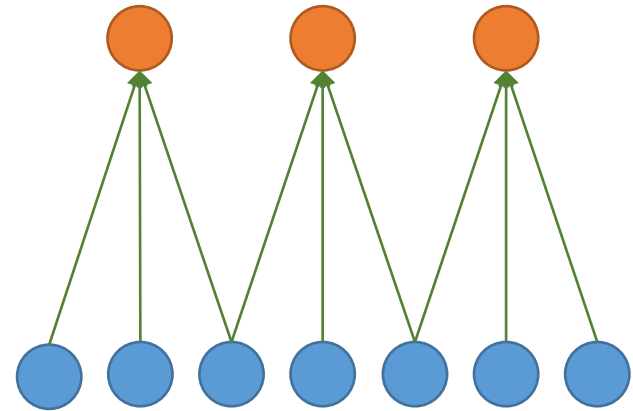
# CNN: Local Connectivity



**Global** connectivity

Hidden layer

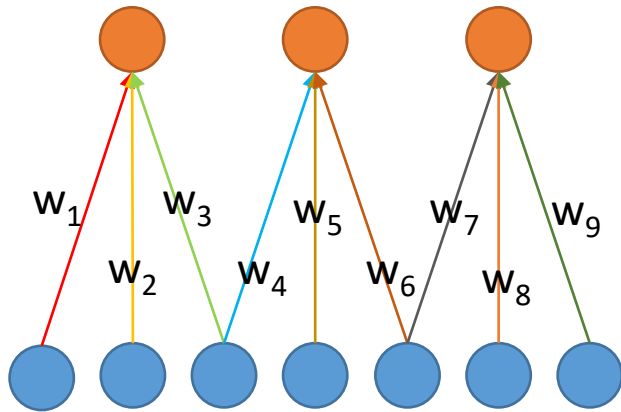
Input layer



**Local** connectivity

- # of input dimensions/units (neurons): 7
- # of output/hidden units: 3
- Number of parameters
  - Global connectivity:
  - Local connectivity:

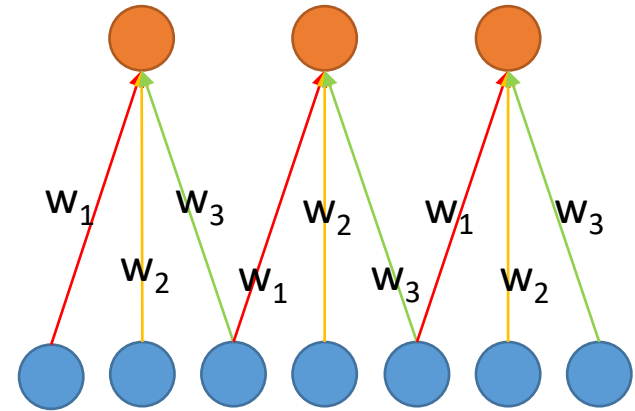
# CNN: Weight Sharing



**Without** weight sharing

Hidden layer

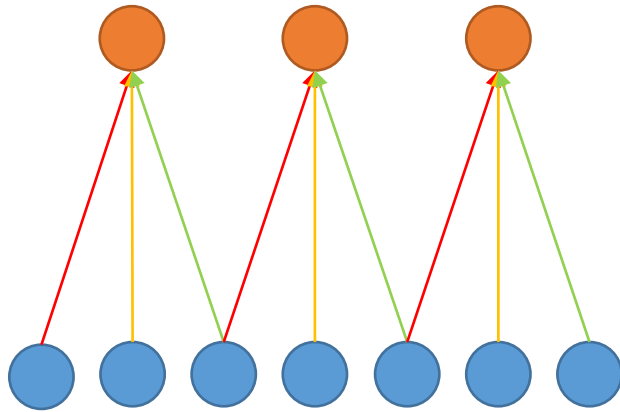
Input layer



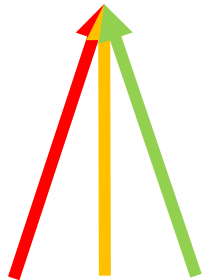
**With** weight sharing

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
  - Without weight sharing:
  - With weight sharing :

# CNN with Multiple Input Channels



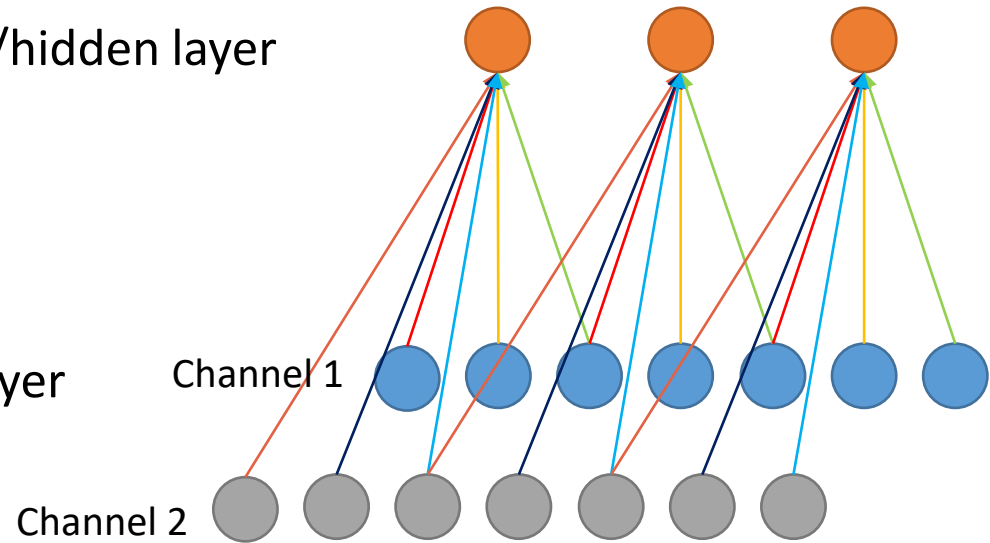
**Single** input channel



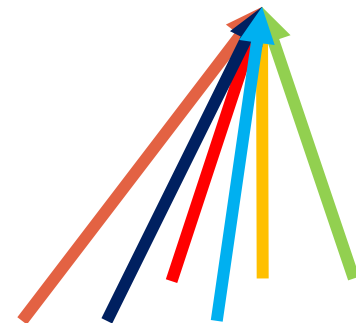
Filter weights

Output/hidden layer

Input layer

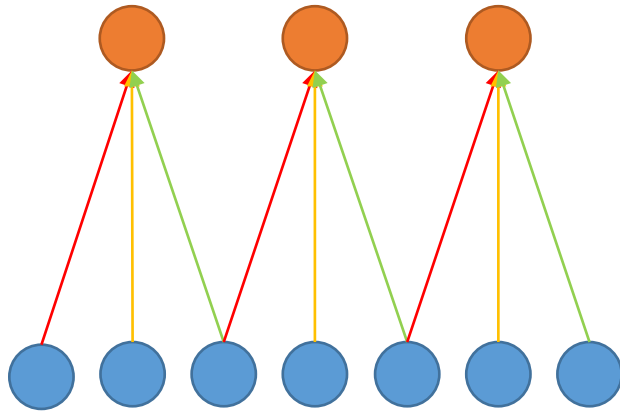


**Multiple** input channels

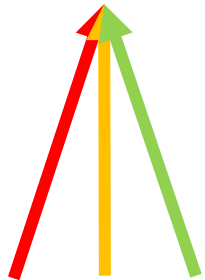


Filter weights

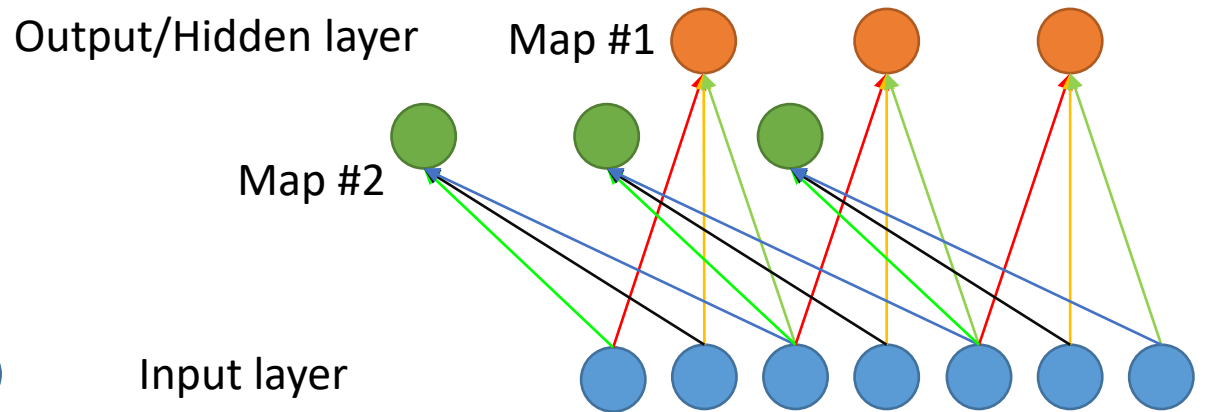
# CNN with Multiple Output Maps



**Single** output map

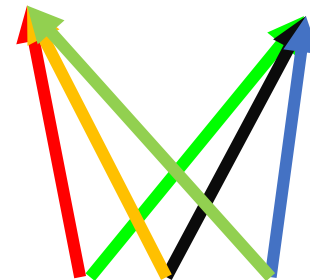


Filter weights



**Multiple** output maps

Filter 1

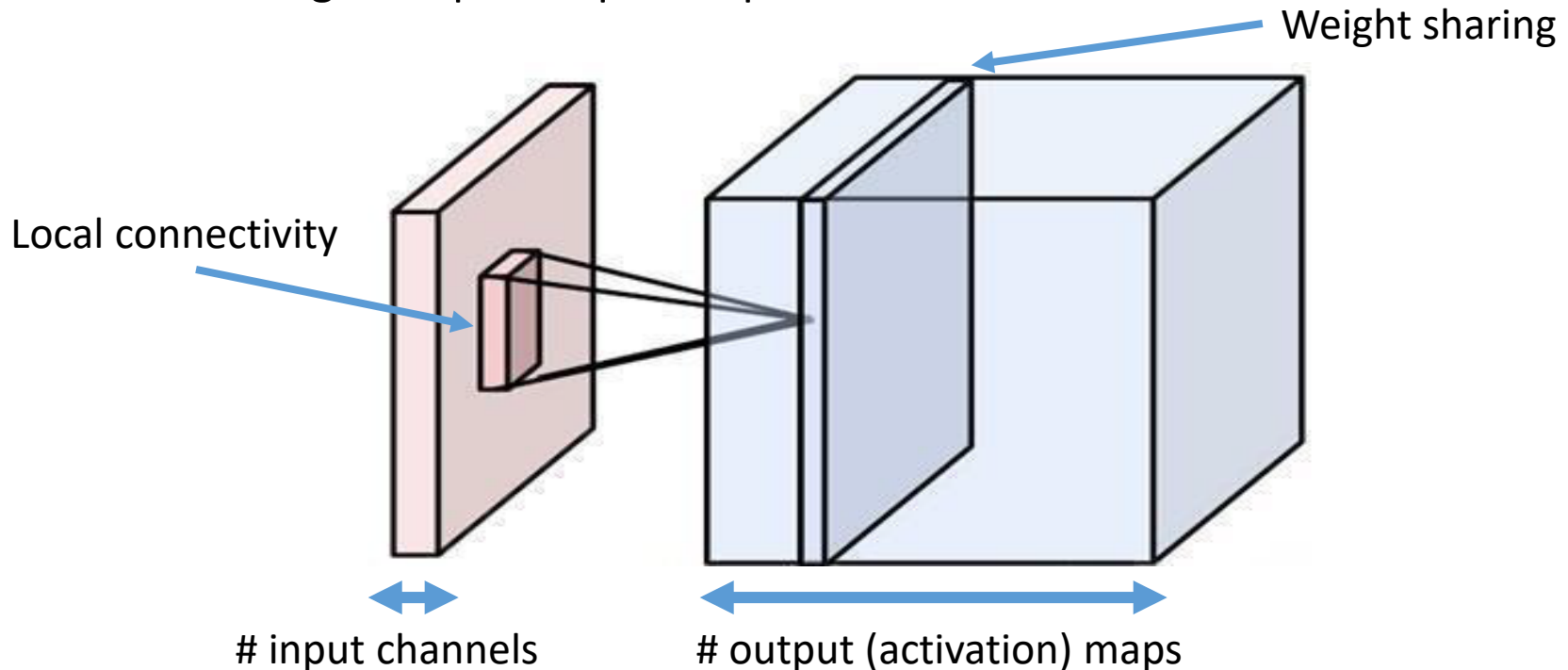


Filter weights

Filter 2

# Putting the ideas together → CNN

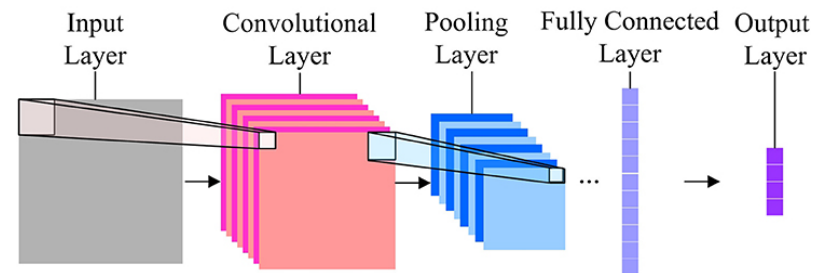
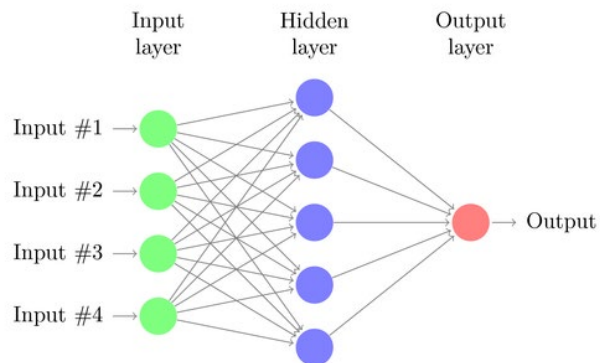
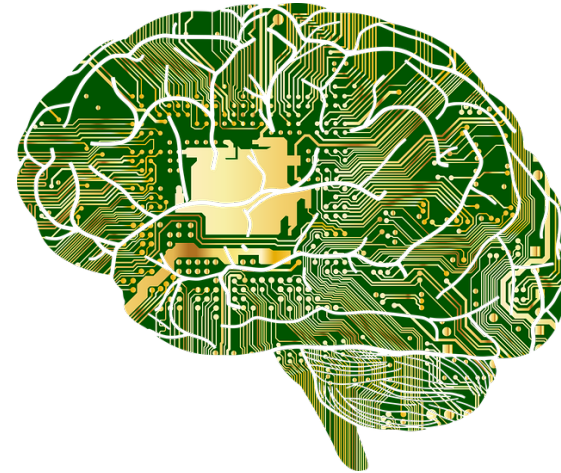
- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps



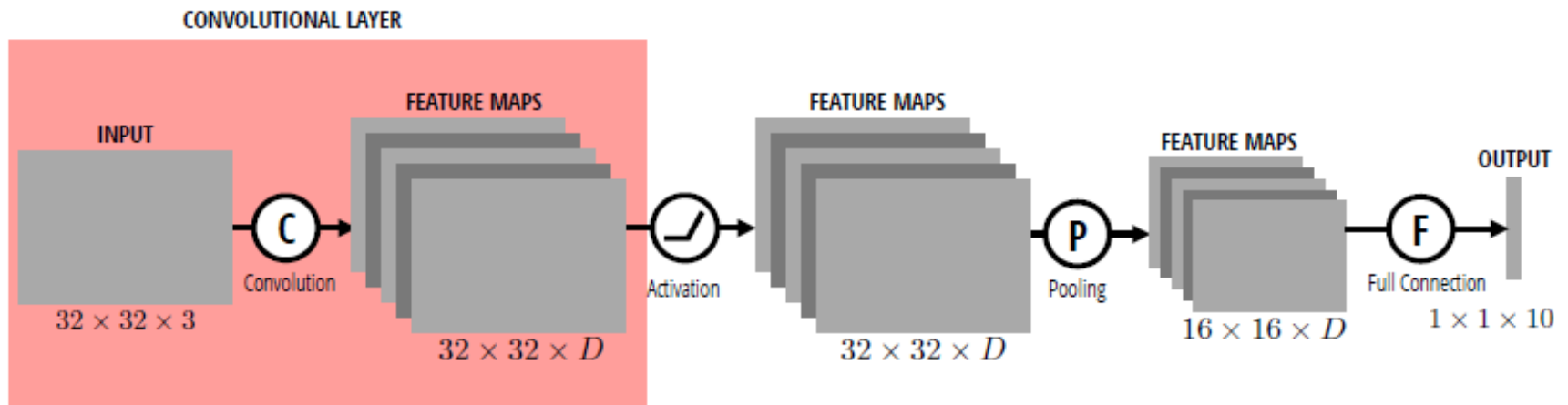


# What's to Be Covered Today...

- Convolution Neural Networks (CNN)
  - Design of CNN
  - Variants of CNNs
  - Training Techniques for CNN



# Convolution Layer in CNN



# What is a Convolution?

- Weighted moving sum



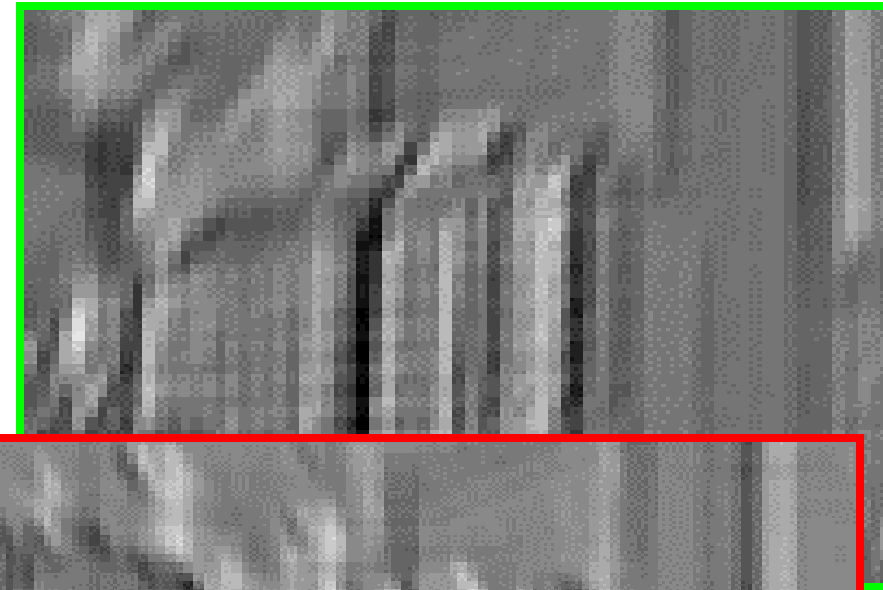
Input



▪  
▪  
▪

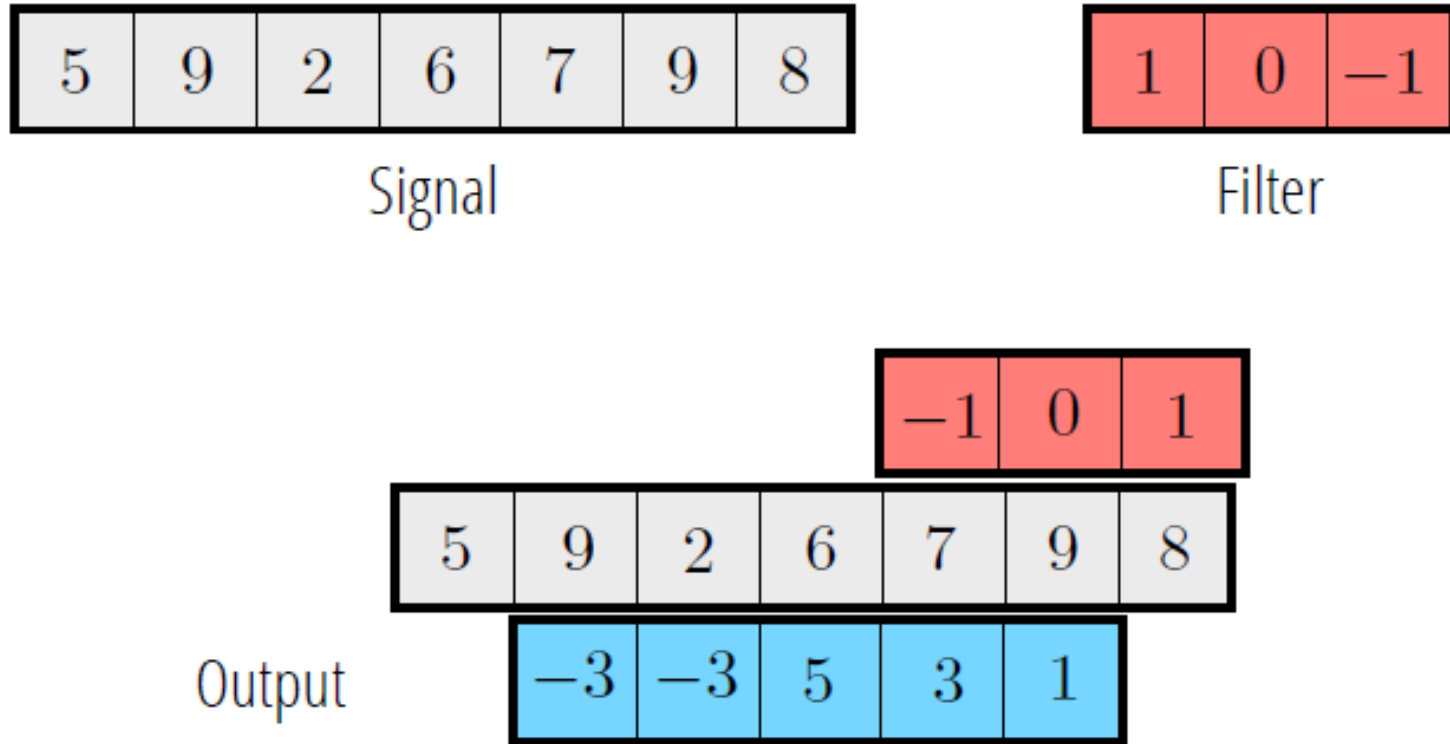


Feature Activation Map #1 14



Feature Activation Map #2

# What is a Convolution?



## Convolution is a local linear operator

# What is a Convolution?

- Toeplitz Matrix Form

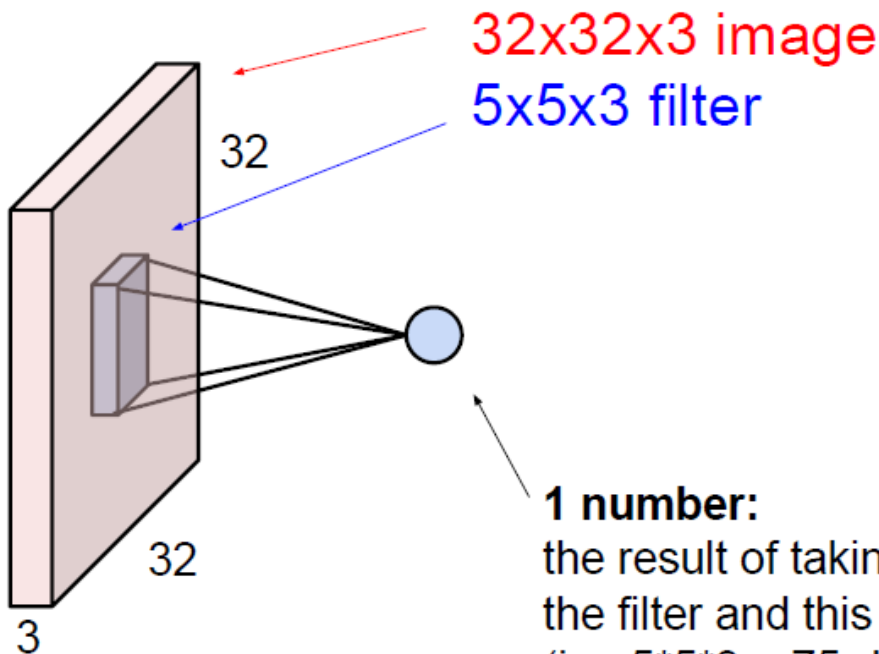
$$\begin{bmatrix}
 \ddots & & & \\
 & w_1 & w_2 & w_3 & & \\
 & & w_1 & w_2 & w_3 & \\
 & & & w_1 & w_2 & w_3 & \\
 & & & & \ddots & \\
 \end{bmatrix}
 \begin{bmatrix}
 x_{11} \\
 x_{12} \\
 \vdots \\
 x_{MN}
 \end{bmatrix}
 +
 \begin{bmatrix}
 b \\
 b \\
 \vdots \\
 b
 \end{bmatrix}
 =
 \begin{bmatrix}
 y_{11} \\
 y_{12} \\
 \vdots \\
 y_{MN}
 \end{bmatrix}$$

Filter

Bias

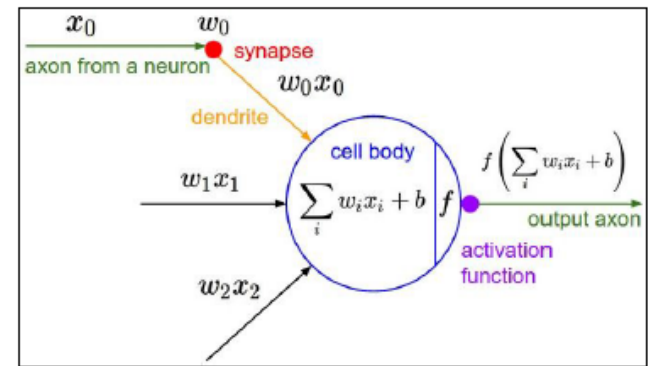
# Putting them together (cont'd)

- The neuron view of a CONV layer



**1 number:**

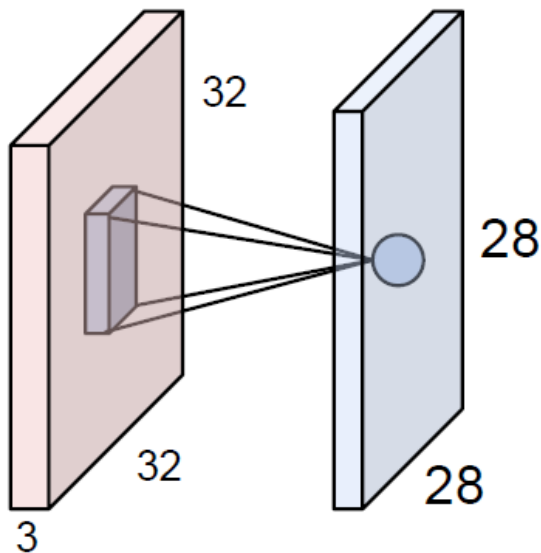
the result of taking a dot product between the filter and this part of the image (i.e.  $5 \times 5 \times 3 = 75$ -dimensional dot product)



It's just a neuron with local connectivity...

# Putting them together (cont'd)

- The neuron view of CONV layer (cont'd)



➤ **Feature map** (at an intermediate layer):

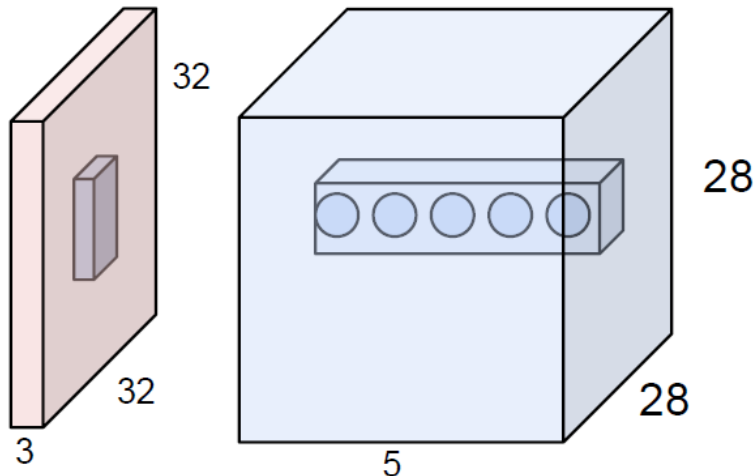
An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

“5x5 filter” -> “5x5 receptive field for each neuron”

# Putting them together (cont'd)

- The neuron view of CONV layer
  - Typically, more than 1 filter is learned in CNN...



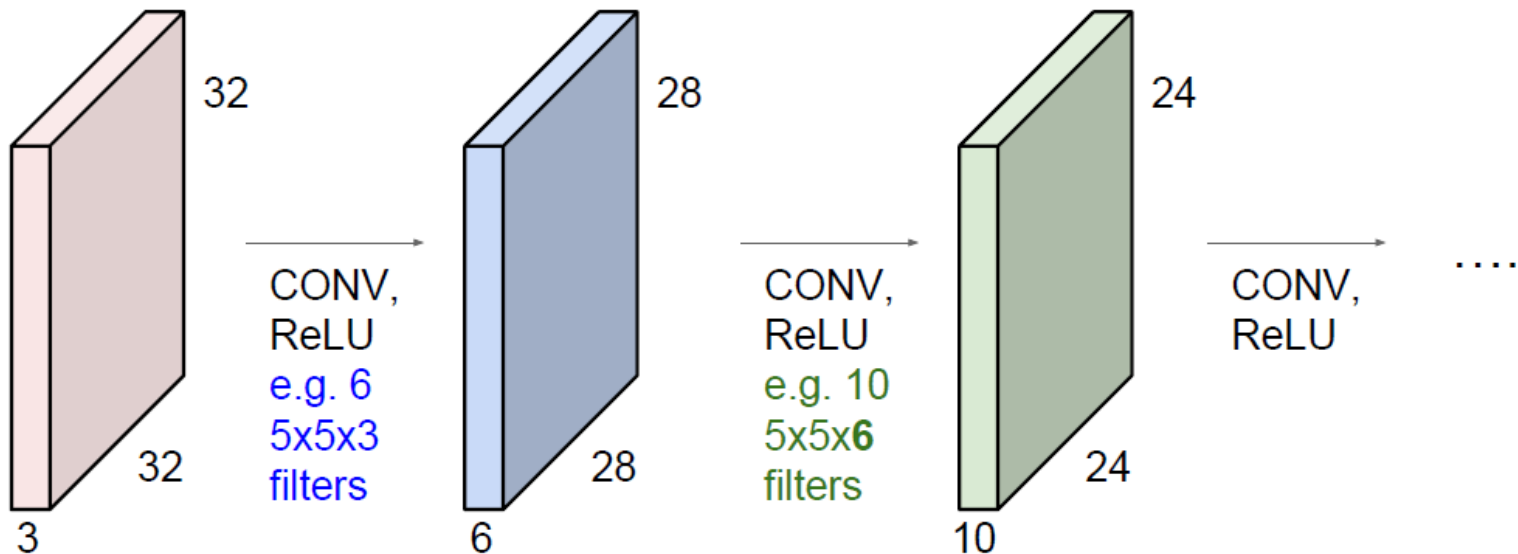
E.g. with 5 filters,  
CONV layer consists of  
neurons arranged in a 3D grid  
(28x28x5)

There will be 5 different  
neurons all looking at the same  
region in the input volume



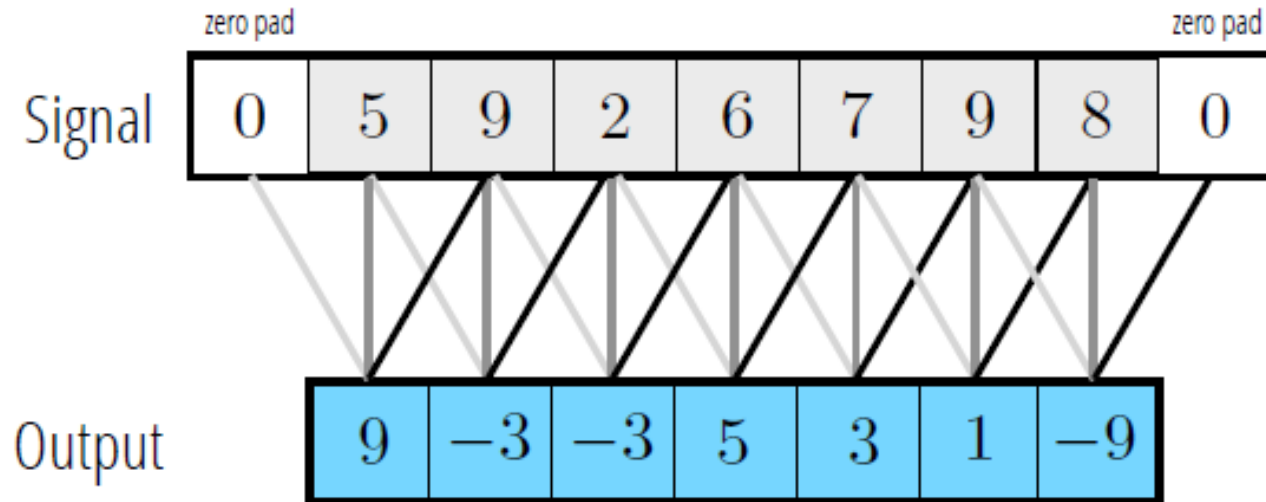
# Putting them together (cont'd)

- Input with 32 x 32 pixels convolved repeatedly w/ 5 x 5 x 3 filters would shrink feature map volumes spatially
  - 32 -> 28 -> 24 -> ...



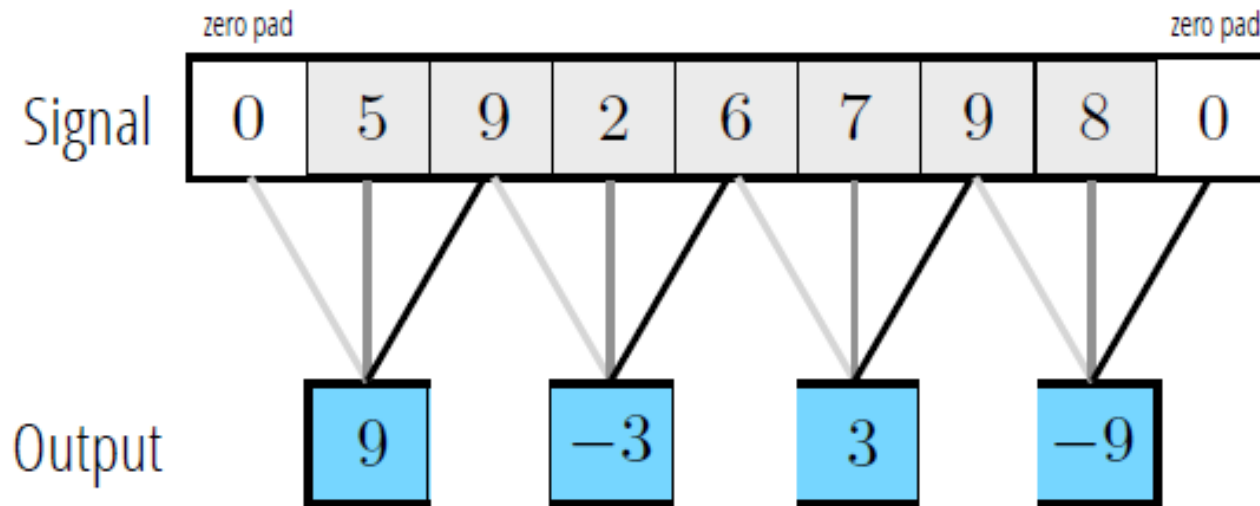
# What is a Convolution? (cont'd)

- Zero Padding
  - Output is the same size as that of the input
    - That is, conv will not shrink as the network gets deeper.



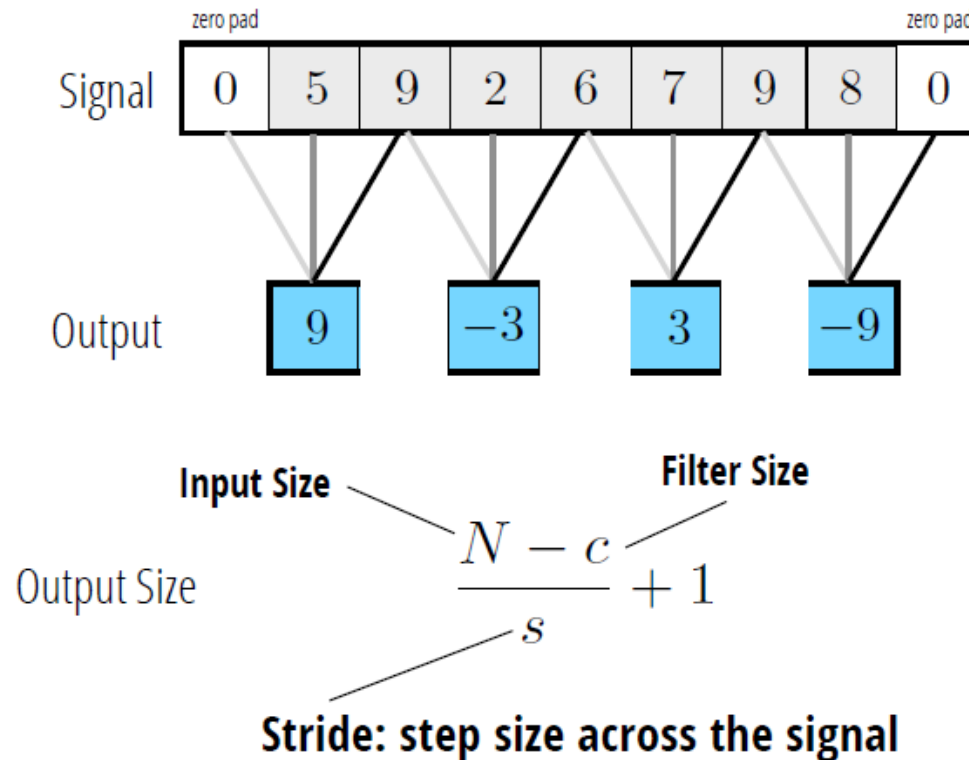
# What is a Convolution? (cont'd)

- Stride
  - Step size across signals
  - Why & when preferable?



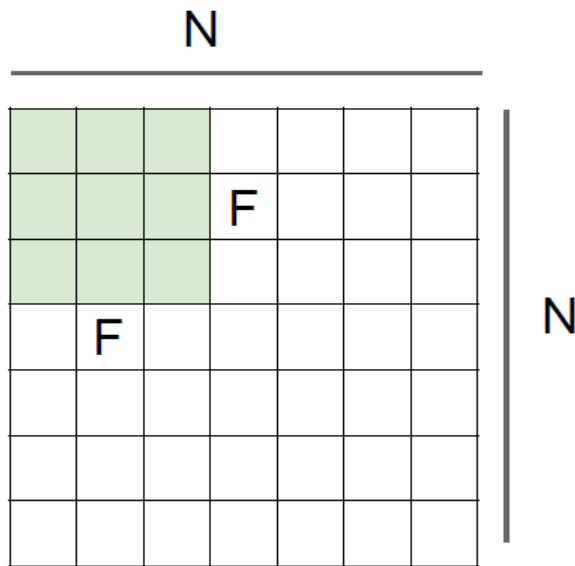
# What is a Convolution? (cont'd)

- Stride
  - Step size across signals
  - See example below:



# What is a Convolution? (cont'd)

- Stride
  - Step size across signals
  - See a 2D example below:



Output size:  
 **$(N - F) / \text{stride} + 1$**

e.g.  $N = 7, F = 3$ :

stride 1  $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2  $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3  $\Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$

# What is a Convolution?

- Zero Padding + Stride

|   |   |   |   |   |   |  |  |  |
|---|---|---|---|---|---|--|--|--|
| 0 | 0 | 0 | 0 | 0 | 0 |  |  |  |
| 0 |   |   |   |   |   |  |  |  |
| 0 |   |   |   |   |   |  |  |  |
| 0 |   |   |   |   |   |  |  |  |
| 0 |   |   |   |   |   |  |  |  |
|   |   |   |   |   |   |  |  |  |
|   |   |   |   |   |   |  |  |  |
|   |   |   |   |   |   |  |  |  |
|   |   |   |   |   |   |  |  |  |

e.g. input 7x7

**3x3** filter, applied with **stride 1**

**pad with 1 pixel** border => what is the output?

in general, common to see CONV layers with  
stride 1, filters of size  $F \times F$ , and zero-padding with  
 $(F-1)/2$ . (will preserve size spatially)

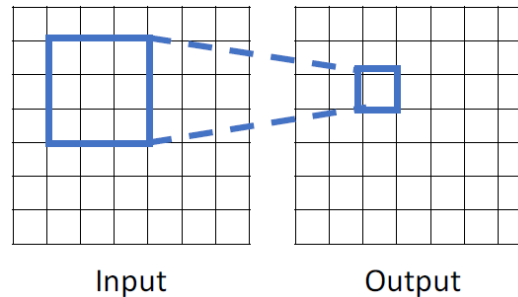
e.g.  $F = 3 \Rightarrow$  zero pad with 1

$F = 5 \Rightarrow$  zero pad with 2

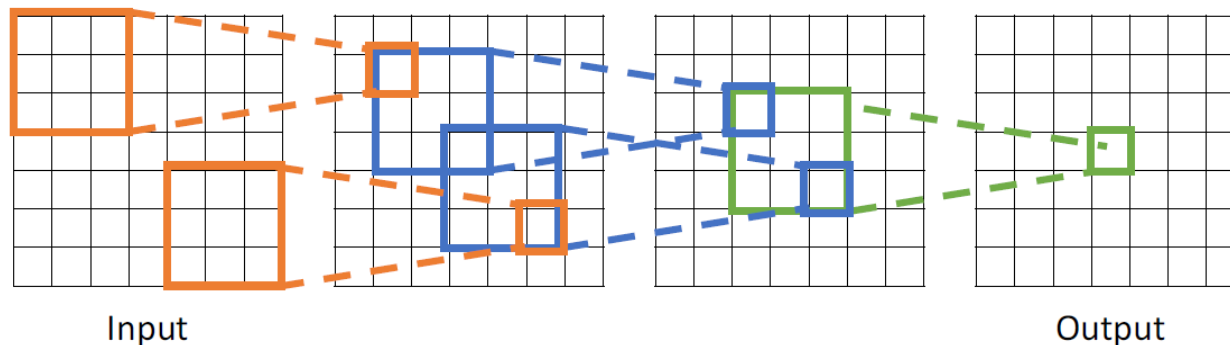
$F = 7 \Rightarrow$  zero pad with 3

# Remarks: Receptive Field

- For convolution with kernel size  $n \times n$ , each entry in the output layer depends on a  $n \times n$  receptive field in the input layer.



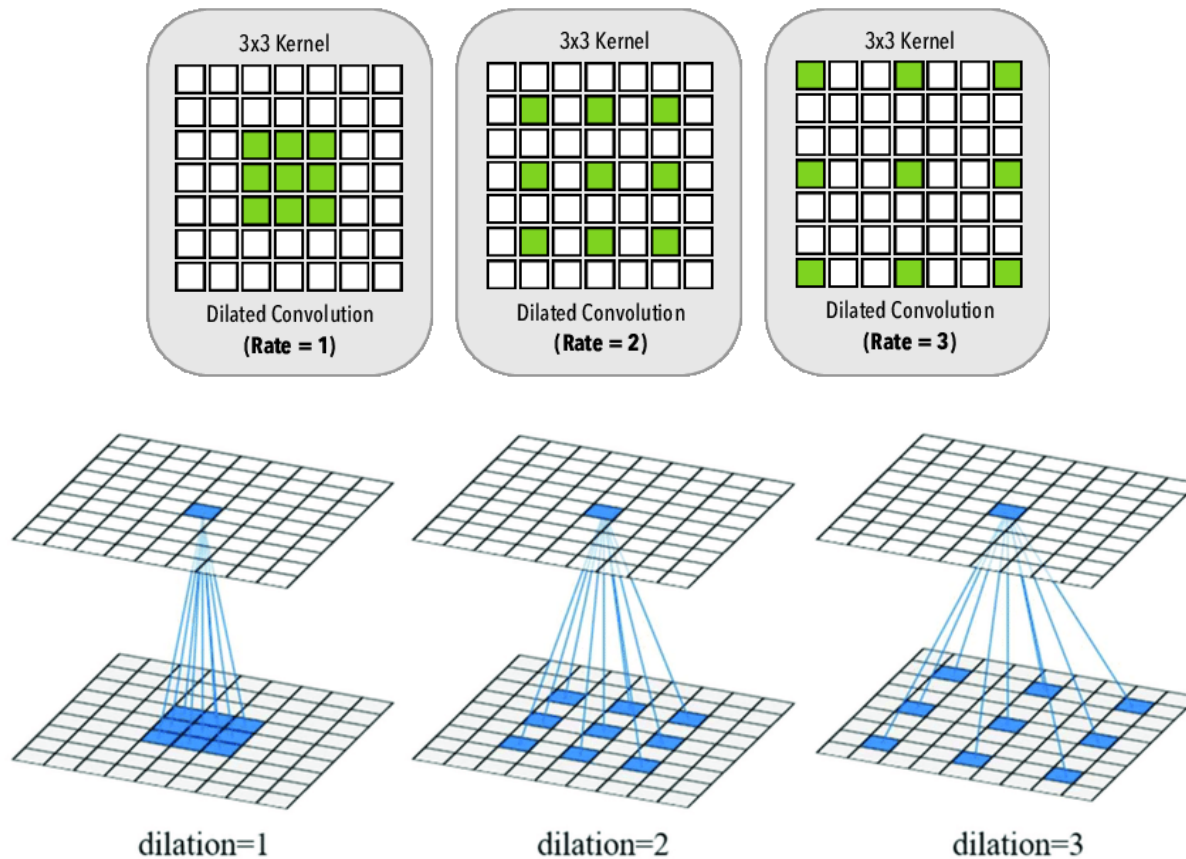
- Each successive convolution adds  $n-1$  to the receptive field size. With a total of  $L$  layers, the receptive field size would be  $1 + (L-1) * (n-1)$ .



- For an image w/ high resolution, we need to deploy multiple CNN layers for the output to “see” the entire input image....**any other alternatives??**

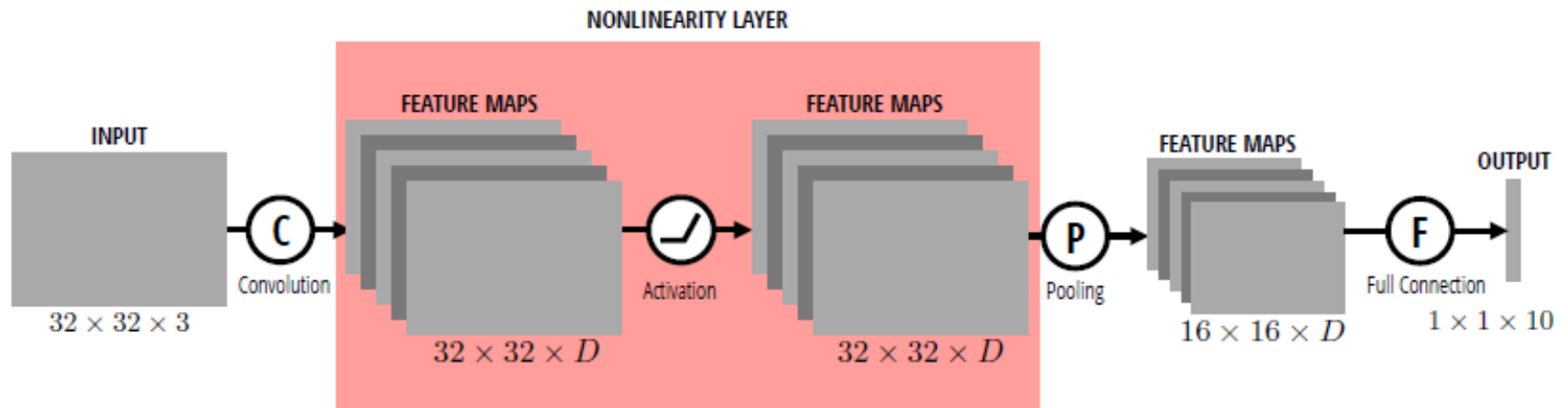
# A Variant of Convolution

- Dilated Convolution
  - Kernel in the same size but capable of handling a larger receptive field



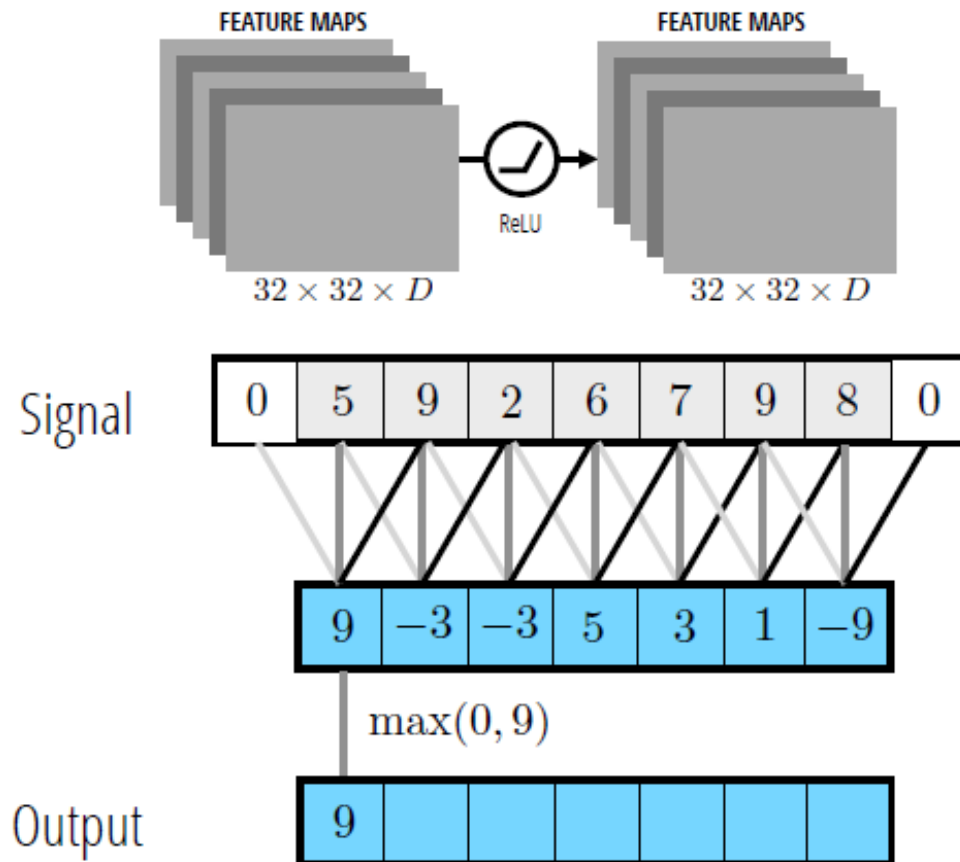


# Nonlinearity Layer in CNN



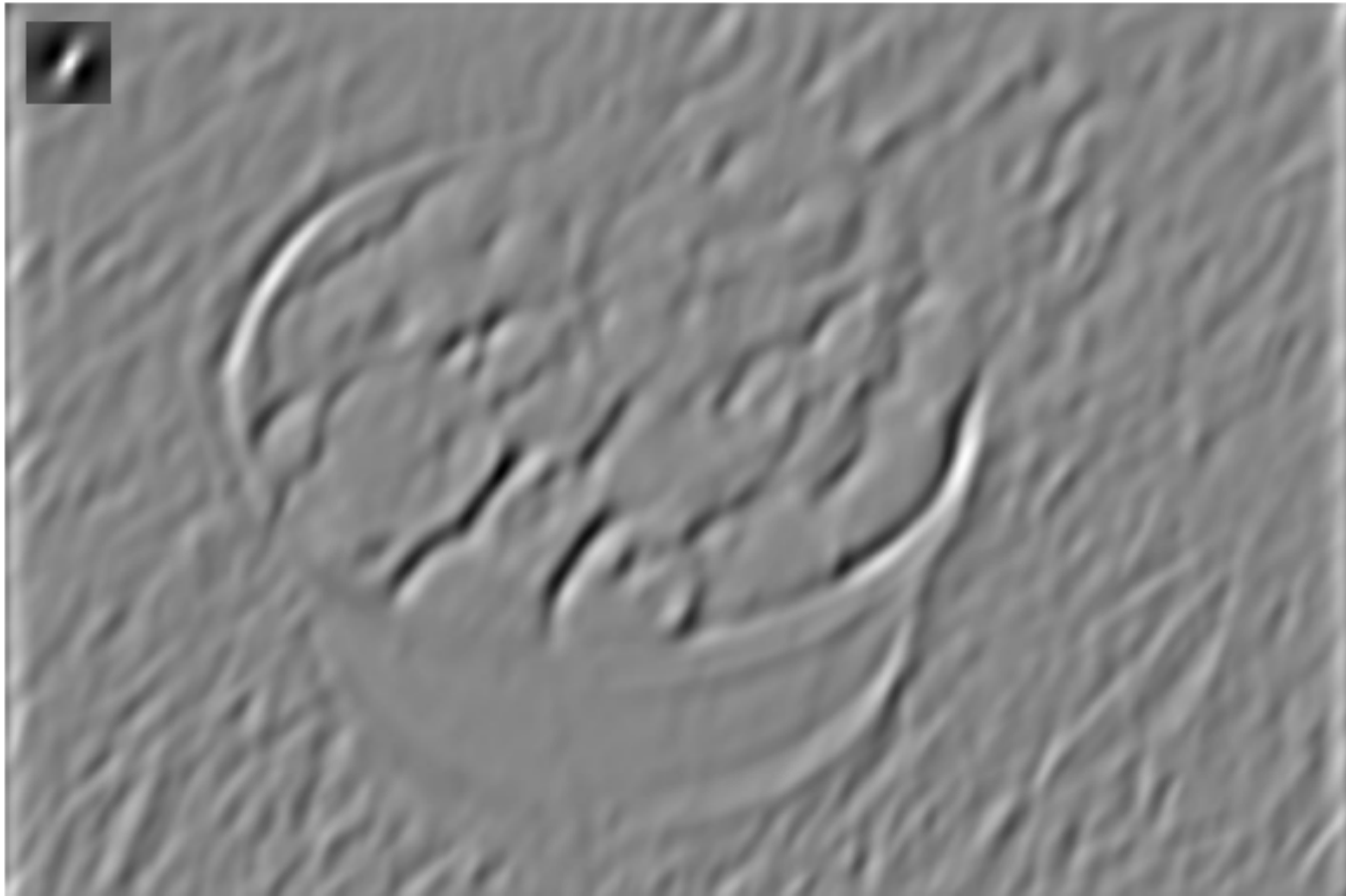
# Nonlinearity Layer

- E.g., ReLU (Rectified Linear Unit)
  - Pixel by pixel computation of  $\max(0, x)$



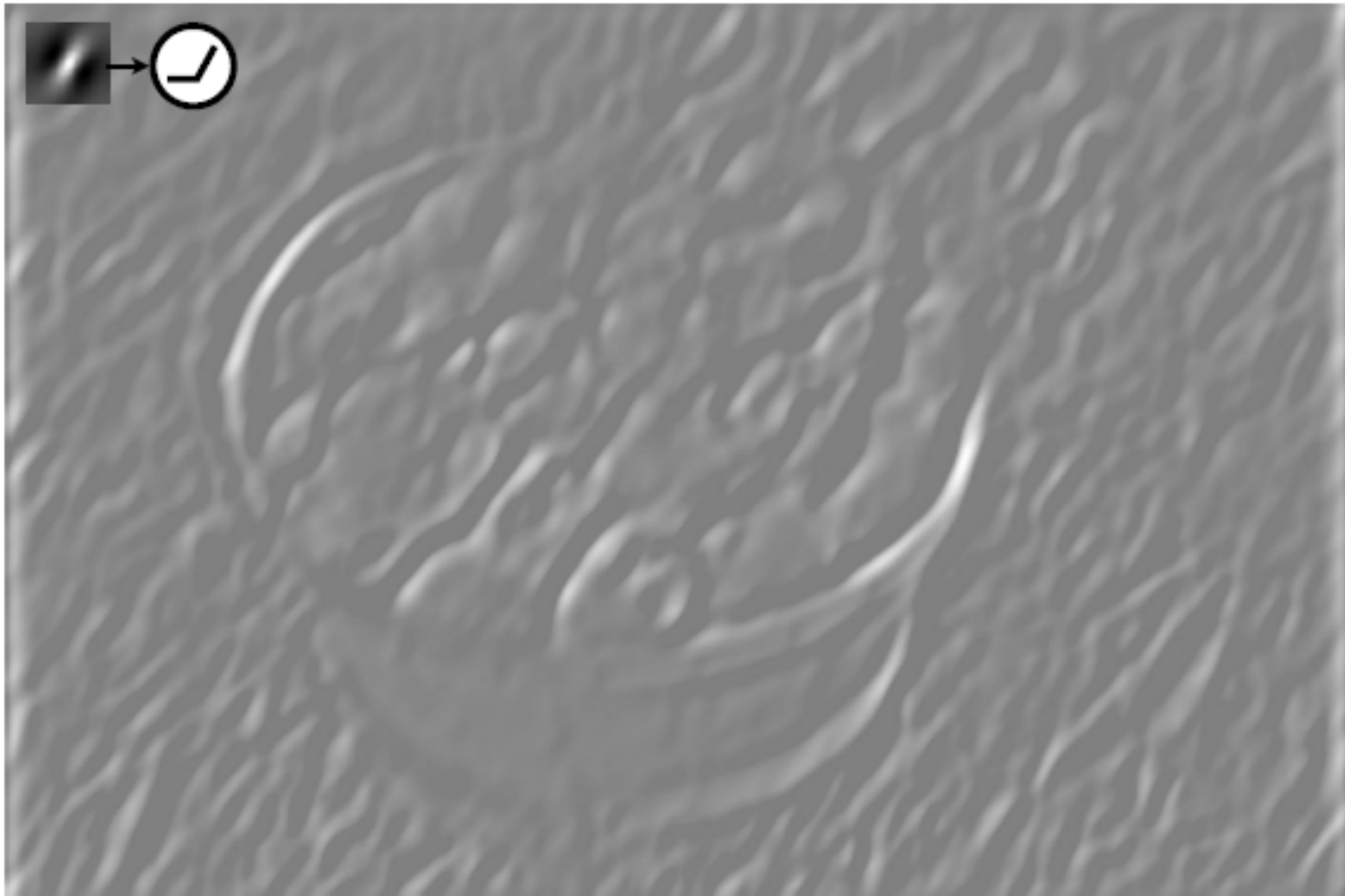
# Nonlinearity Layer

- E.g., ReLU (Rectified Linear Unit)
  - Pixel by pixel computation of  $\max(0, x)$

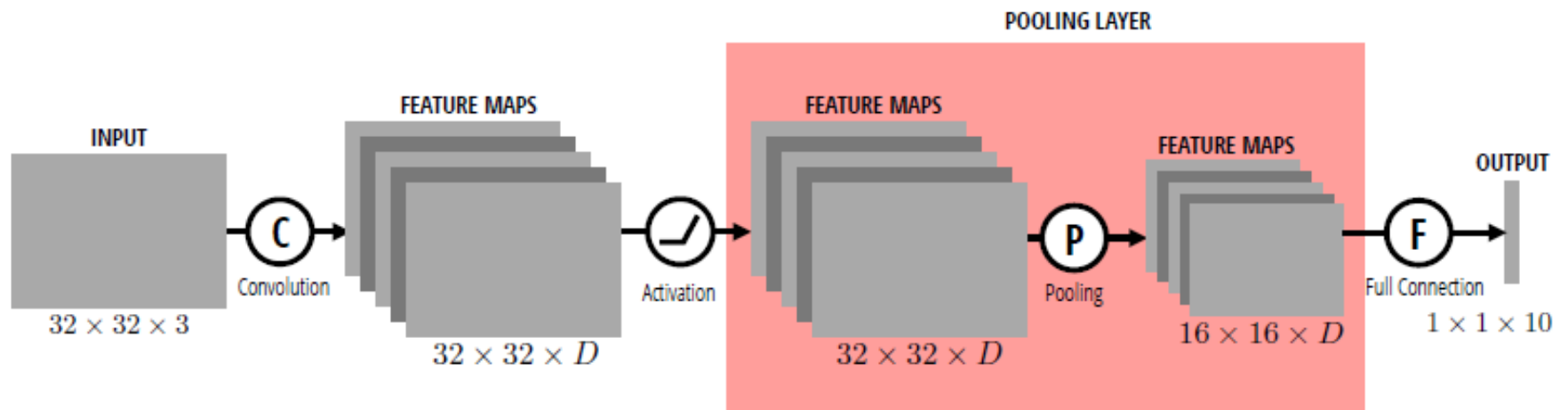


# Nonlinearity Layer

- E.g., ReLU (Rectified Linear Unit)
  - Pixel by pixel computation of  $\max(0, x)$

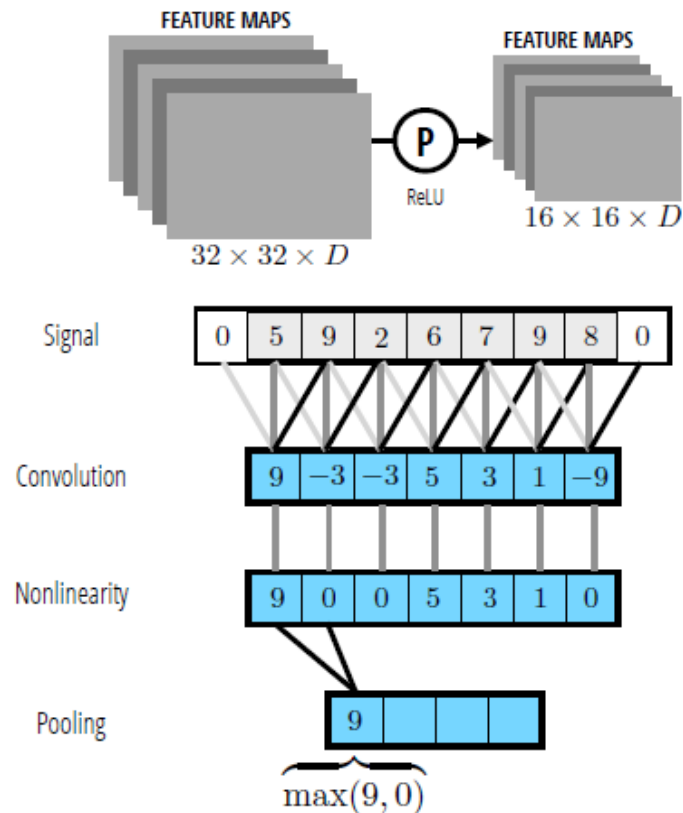


# Pooling Layer in CNN



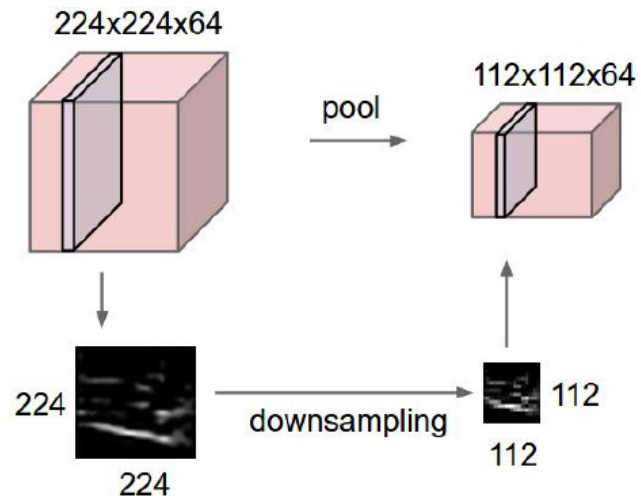
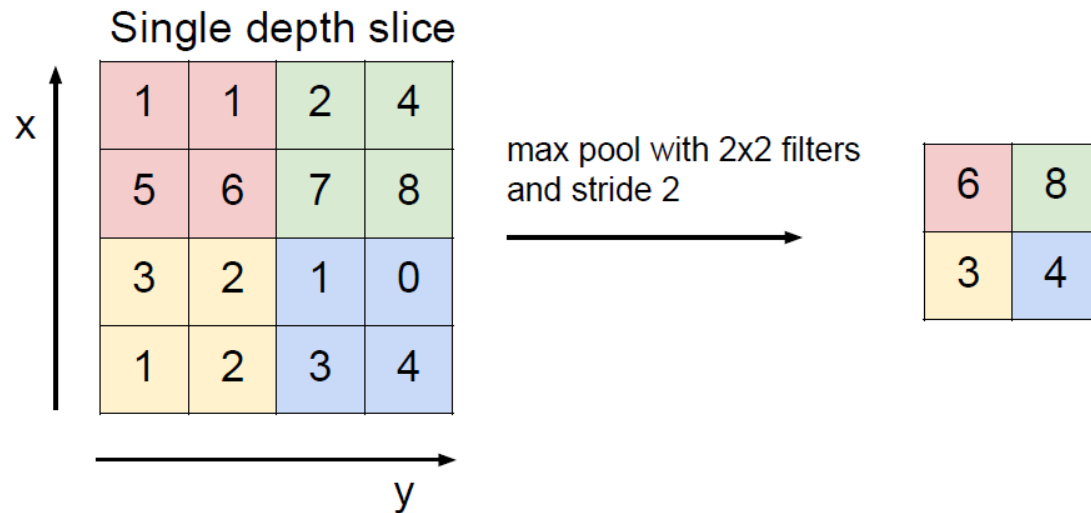
# Pooling Layer

- Makes the representations smaller and more manageable
- Operates over each activation map independently
- E.g., Max Pooling

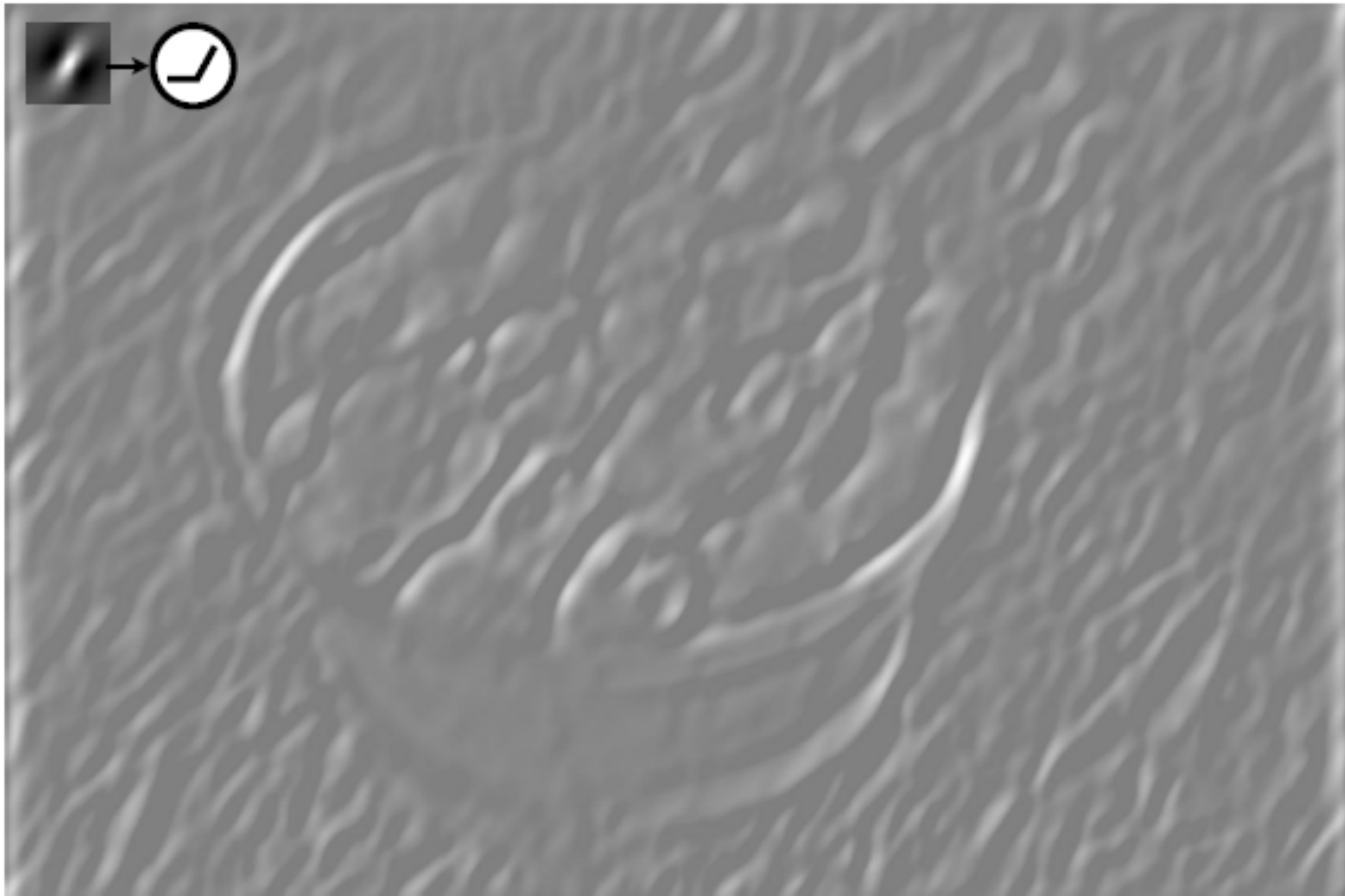


# Pooling Layer

- Reduces the spatial size and provides spatial invariance

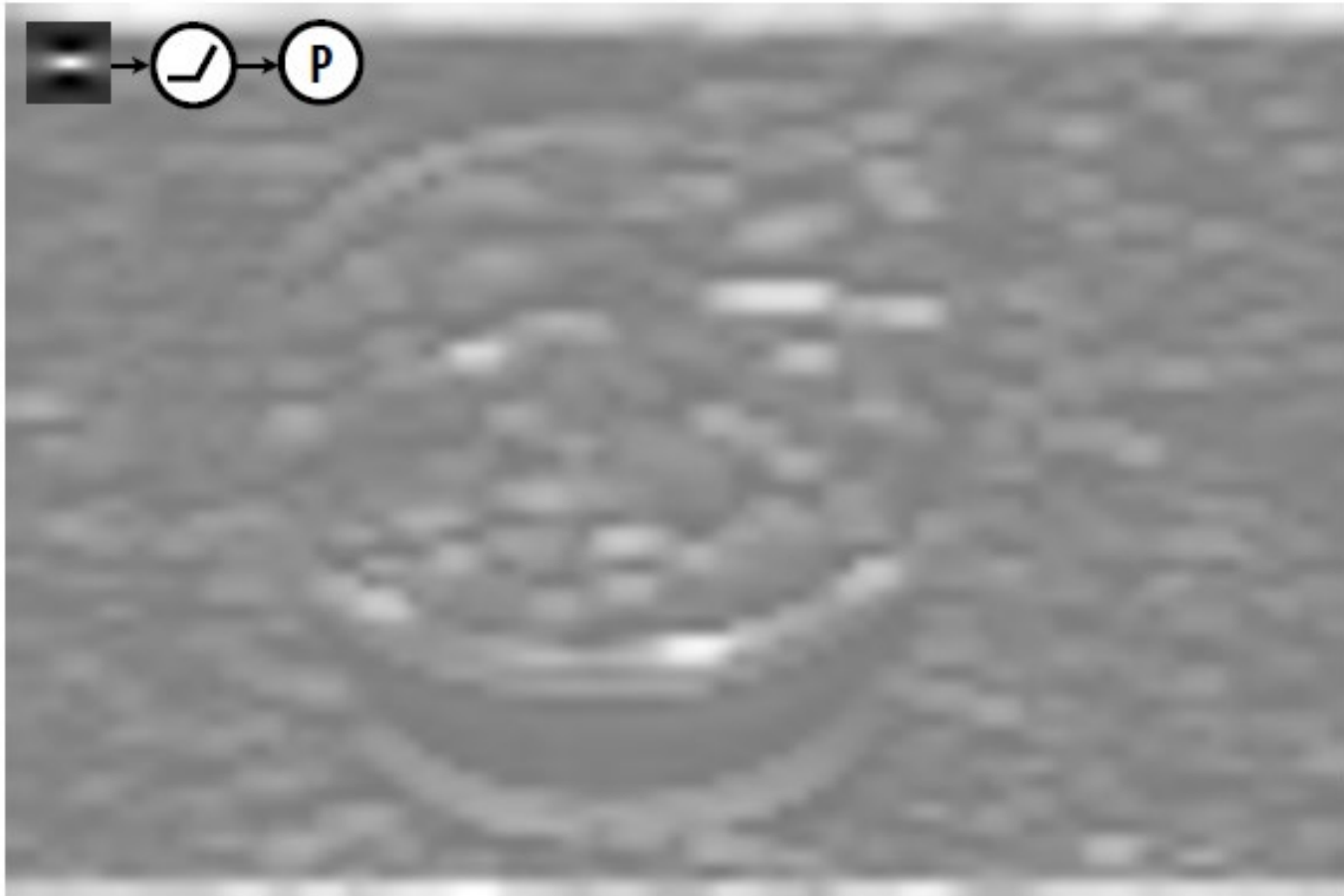


- Example
  - Nonlinearity by ReLU

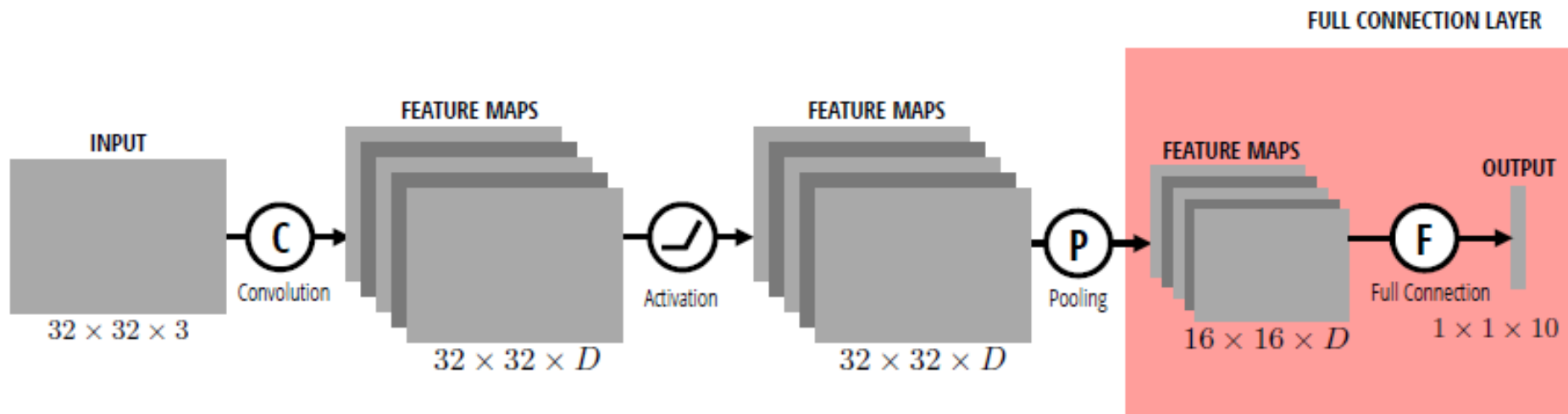




- Example
  - Max pooling

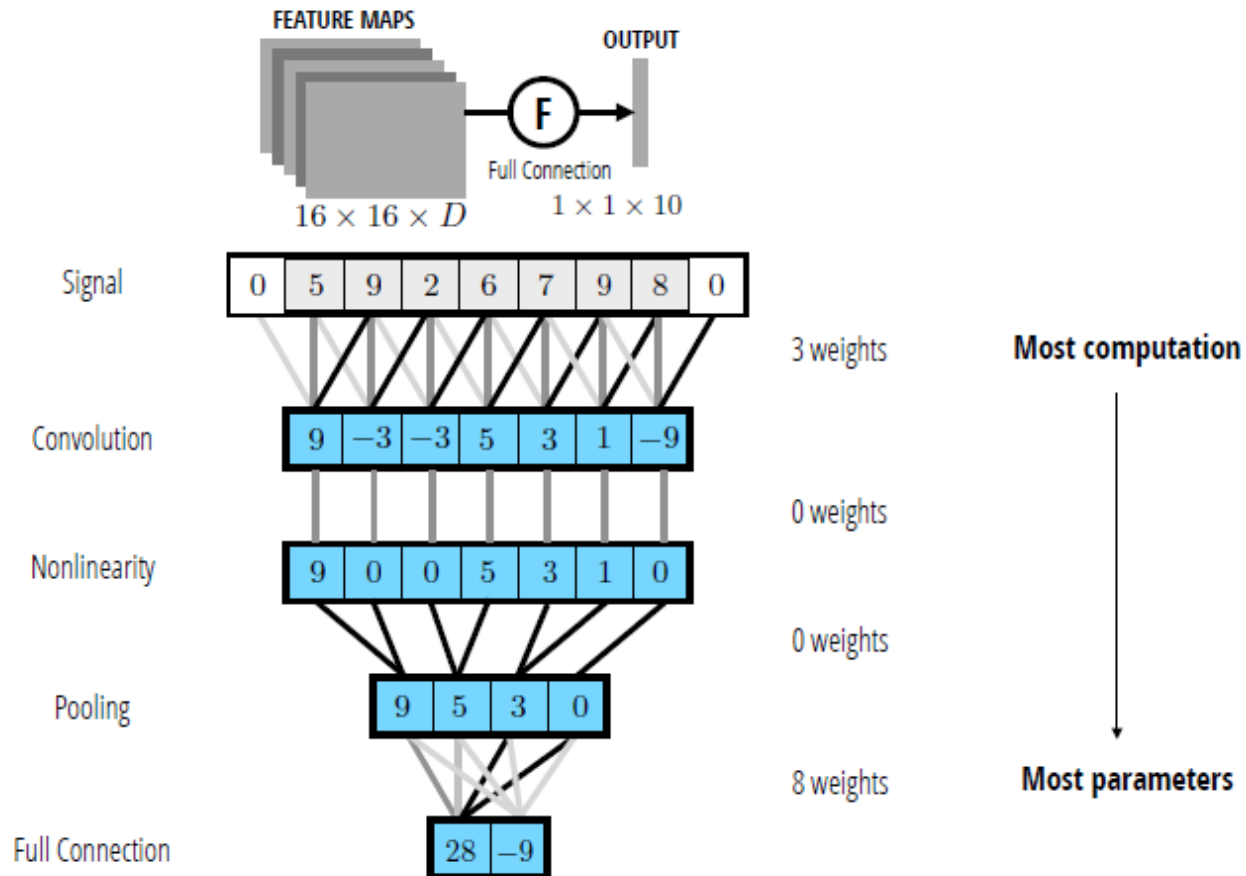


# Fully Connected (FC) Layer in CNN



# FC Layer

- Mapping features/neurons that connect to the entire input volume to the desirable output (e.g., predicted scores for each class)



# CNN

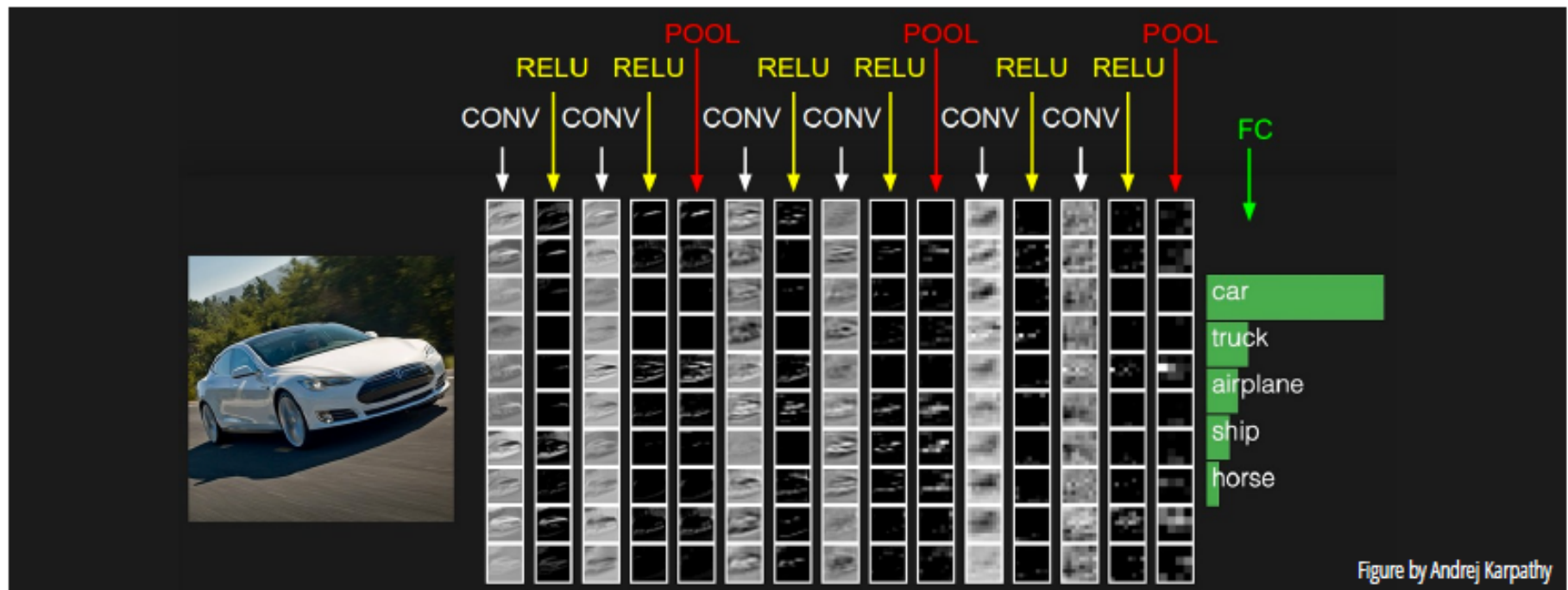
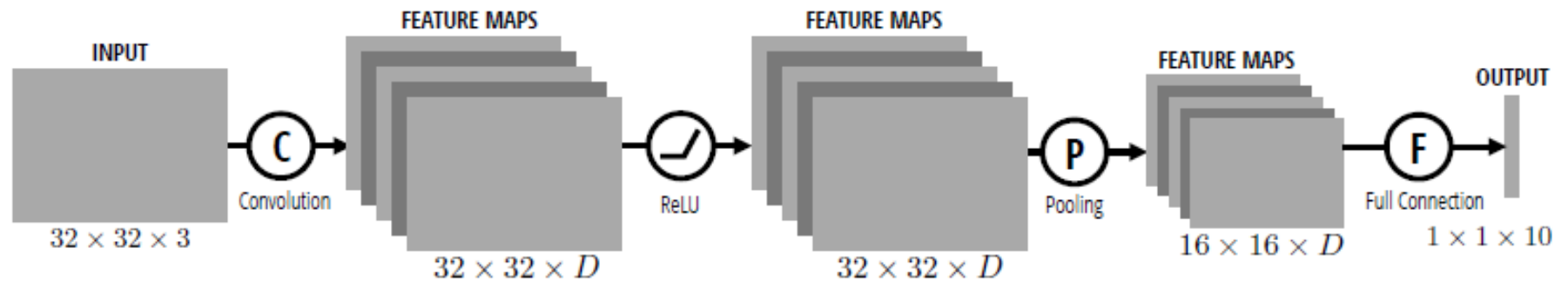
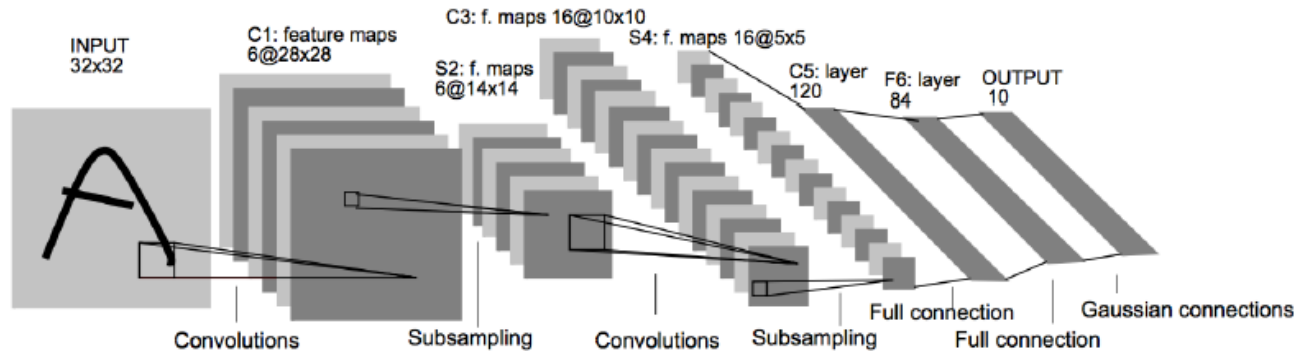


Figure by Andrej Karpathy

# LeNet

- By Yann LeCun; reading digits in 1990s
- Has the elements of modern CNN architectures



# AlexNet [Krizhevsky et al., 2012]

- Repopularized CNN by winning the ImageNet Challenge 2012
- 7 hidden layers, 650,000 neurons, 60M parameters
- Error rate of 16% vs. 26% for 2<sup>nd</sup> place.

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

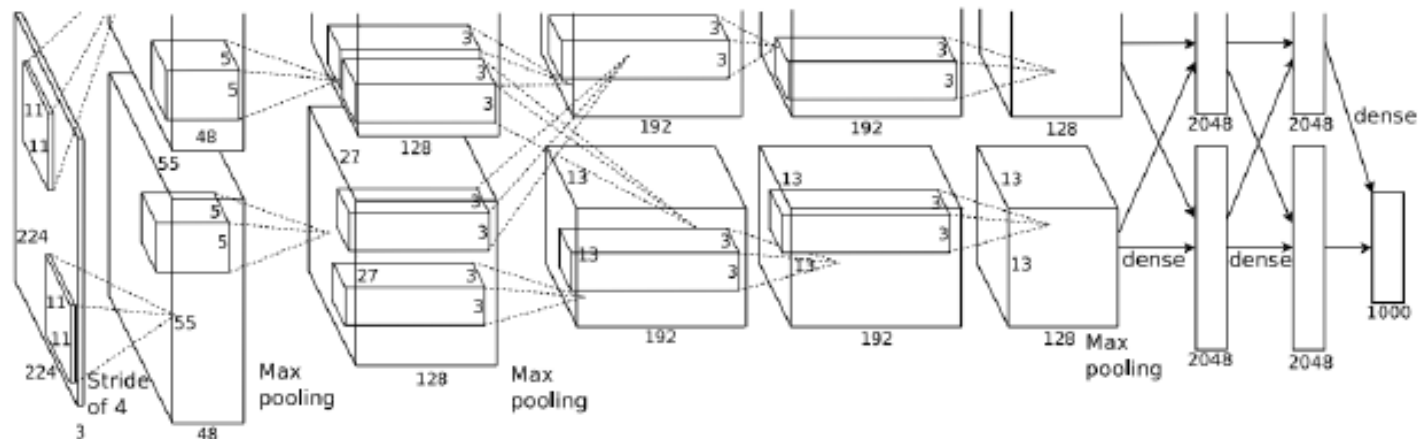
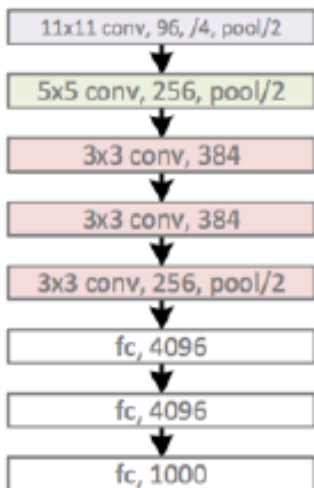
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

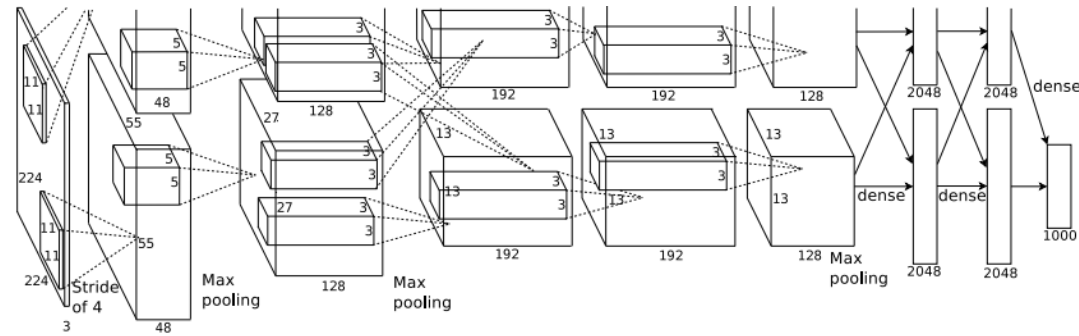
[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



# # of Hyperparameters in AlexNet (cont'd)



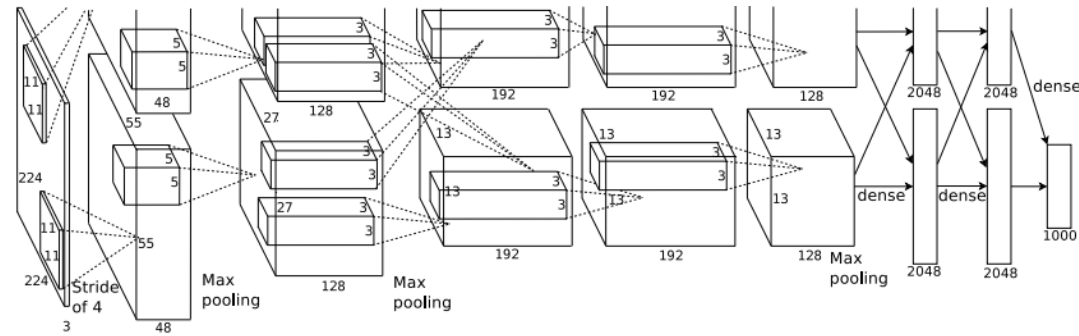
| Layer | Input size |       | Layer   |        |        |     | Output size |       | memory (KB) |
|-------|------------|-------|---------|--------|--------|-----|-------------|-------|-------------|
|       | C          | H / W | filters | kernel | stride | pad | C           | H / W |             |
| conv1 | 3          | 227   | 64      | 11     | 4      | 2   | 64          | 56    | 784         |

$$\begin{aligned}\text{Number of output elements} &= C * H' * W' \\ &= 64 * 56 * 56 = 200,704\end{aligned}$$

$$\text{Bytes per element} = 4 \text{ (for 32-bit floating point)}$$

$$\begin{aligned}\text{KB} &= (\text{number of elements}) * (\text{bytes per elem}) / 1024 \\ &= 200704 * 4 / 1024 \\ &= \mathbf{784}\end{aligned}$$

# # of Hyperparameters in AlexNet (cont'd)



| Layer | Input size |       | Layer   |        |        |     | Output size |       | memory (KB) | params (k) |
|-------|------------|-------|---------|--------|--------|-----|-------------|-------|-------------|------------|
|       | C          | H / W | filters | kernel | stride | pad | C           | H / W |             |            |
| conv1 | 3          | 227   | 64      | 11     | 4      | 2   | 64          | 56    | 784         | 23         |

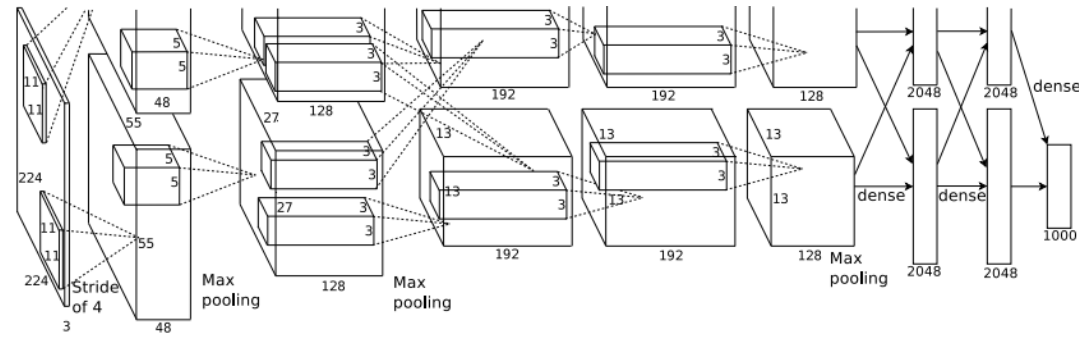
$$\begin{aligned}\text{Weight shape} &= C_{\text{out}} \times C_{\text{in}} \times K \times K \\ &= 64 \times 3 \times 11 \times 11\end{aligned}$$

$$\text{Bias shape} = C_{\text{out}} = 64$$

$$\begin{aligned}\text{Number of weights} &= 64 \times 3 \times 11 \times 11 + 64 \\ &= \mathbf{23,296}\end{aligned}$$



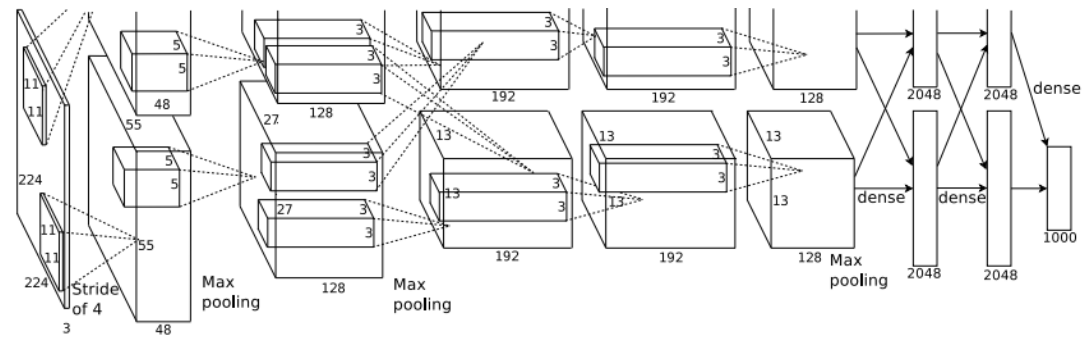
# # of Hyperparameters in AlexNet (cont'd)



|       | Input size |       | Layer   |        |        |     | Output size |       |             |            |          |
|-------|------------|-------|---------|--------|--------|-----|-------------|-------|-------------|------------|----------|
| Layer | C          | H / W | filters | kernel | stride | pad | C           | H / W | memory (KB) | params (k) | flop (M) |
| conv1 | 3          | 227   | 64      | 11     | 4      | 2   | 64          | 56    | 784         | 23         | 73       |

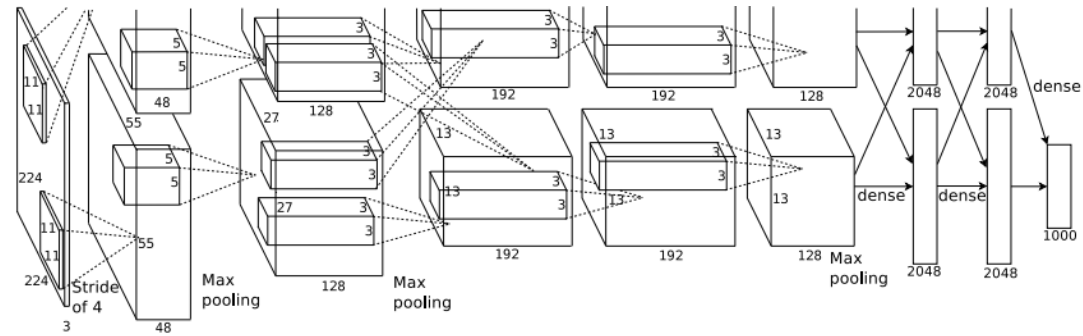
Number of floating point operations (multiply+add)  
 = (number of output elements) \* (ops per output elem)  
 =  $(C_{out} \times H' \times W') \times (C_{in} \times K \times K)$   
 =  $(64 \times 56 \times 56) \times (3 \times 11 \times 11)$   
 =  $200,704 \times 363$   
 = **72,855,552**

# # of Hyperparameters in AlexNet (cont'd)

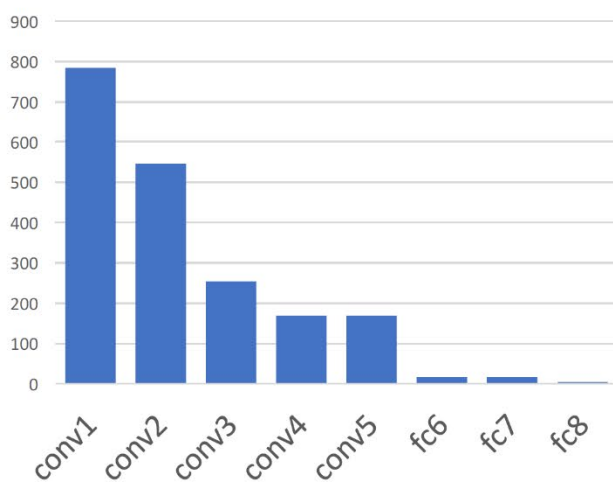


|         | Input size |       | Layer   |        |        |     | Output size |       |             |            |          |
|---------|------------|-------|---------|--------|--------|-----|-------------|-------|-------------|------------|----------|
| Layer   | C          | H / W | filters | kernel | stride | pad | C           | H / W | memory (KB) | params (k) | flop (M) |
| conv1   | 3          | 227   | 64      | 11     | 4      | 2   | 64          | 56    | 784         | 23         | 73       |
| pool1   | 64         | 56    |         | 3      | 2      | 0   | 64          | 27    | 182         | 0          | 0        |
| conv2   | 64         | 27    | 192     | 5      | 1      | 2   | 192         | 27    | 547         | 307        | 224      |
| pool2   | 192        | 27    |         | 3      | 2      | 0   | 192         | 13    | 127         | 0          | 0        |
| conv3   | 192        | 13    | 384     | 3      | 1      | 1   | 384         | 13    | 254         | 664        | 112      |
| conv4   | 384        | 13    | 256     | 3      | 1      | 1   | 256         | 13    | 169         | 885        | 145      |
| conv5   | 256        | 13    | 256     | 3      | 1      | 1   | 256         | 13    | 169         | 590        | 100      |
| pool5   | 256        | 13    |         | 3      | 2      | 0   | 256         | 6     | 36          | 0          | 0        |
| flatten | 256        | 6     |         |        |        |     | 9216        |       | 36          | 0          | 0        |
| fc6     | 9216       |       | 4096    |        |        |     | 4096        |       | 16          | 37,749     | 38       |
| fc7     | 4096       |       | 4096    |        |        |     | 4096        |       | 16          | 16,777     | 17       |
| fc8     | 4096       |       | 1000    |        |        |     | 1000        |       | 4           | 4,096      | 4        |

# Additional Remarks on AlexNet

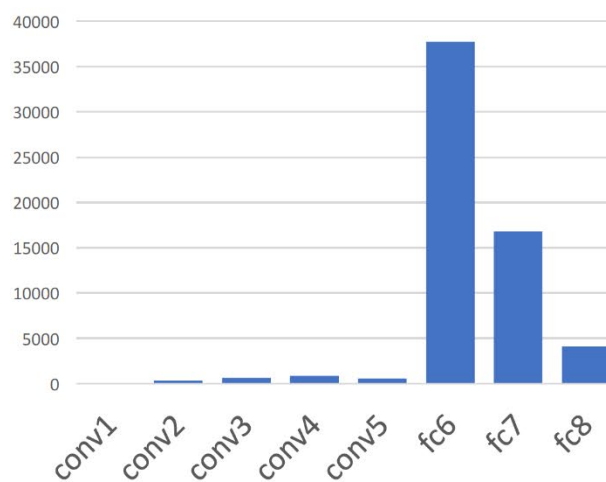


Memory (KB)



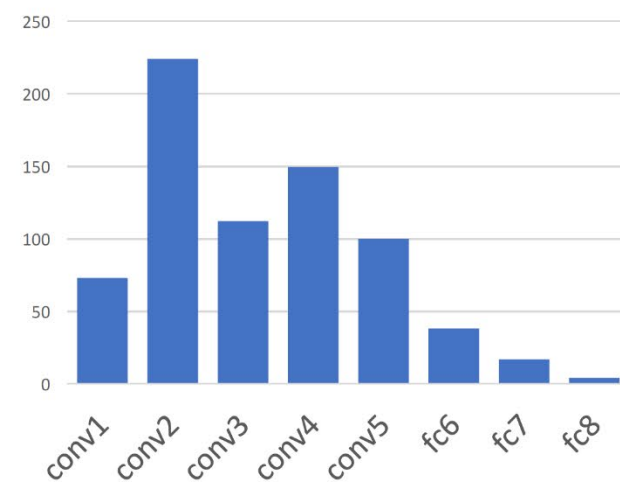
Most of the memory usage in early convolution layers

Params (K)



Nearly all the parameters are in the fully connected layers

MFLOP



Most floating-point operations occur in the convolution layers

# Deep or Not?

- Depth of the network is critical for performance.



**AlexNet:** 8 Layers with 18.2% top-5 error

**Removing Layer 7** reduces 16 million parameters, but only 1.1% drop in performance!

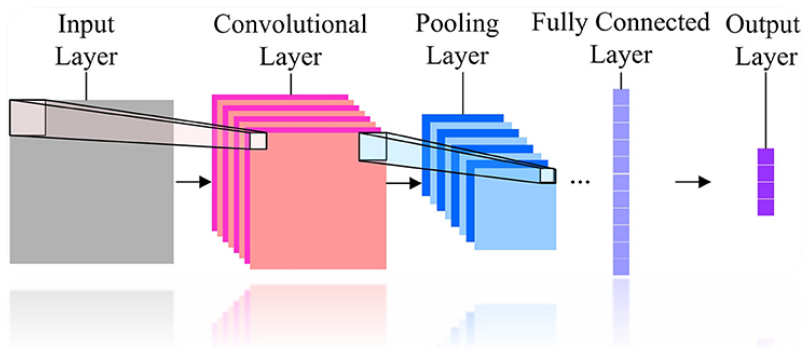
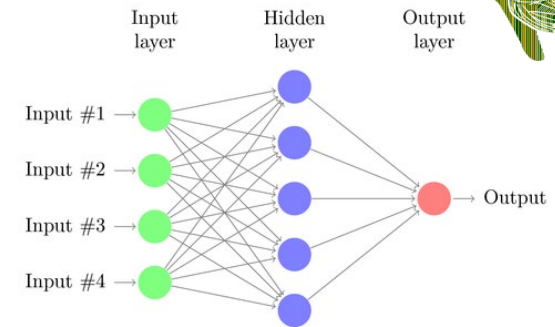
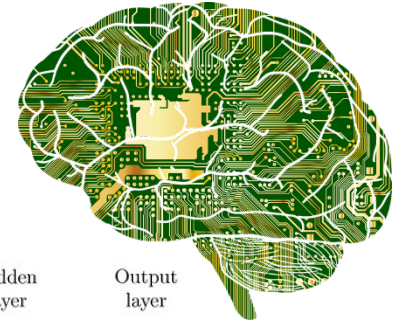
**Removing Layer 6 and 7** reduces 50 million parameters, but only 5.7% drop in performance

**Removing middle conv layers** reduces 1 million parameters, but only 3% drop in performance

**Removing feature & conv layers** produces a **33% drop** in performance

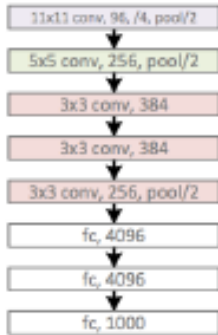
# What to Cover Today...

- Convolution Neural Networks (CNN)
  - Design of CNN
  - Variants of CNNs
  - Training Techniques for CNN
- Image Segmentation

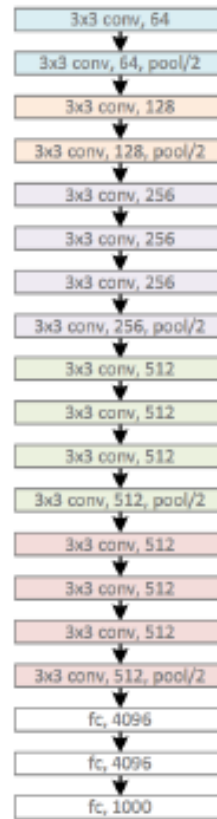


# CNN: A Revolution of Depth

AlexNet, 8 layers  
(ILSVRC 2012)



VGG, 19 layers  
(ILSVRC 2014)

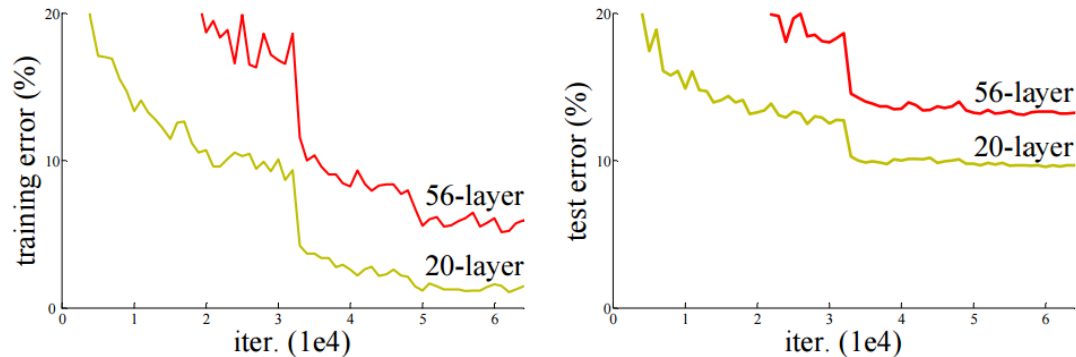


GoogleNet, 22 layers  
(ILSVRC 2014)

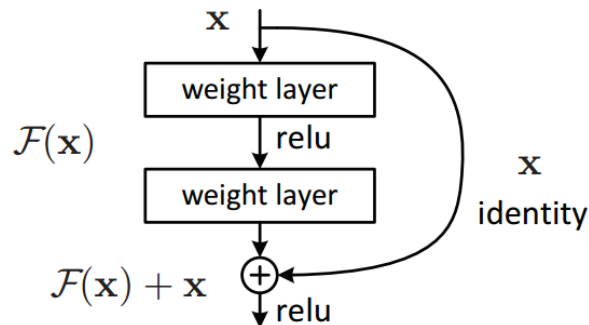


# ResNet

- Can we just increase the #layer? What are the potential risks?



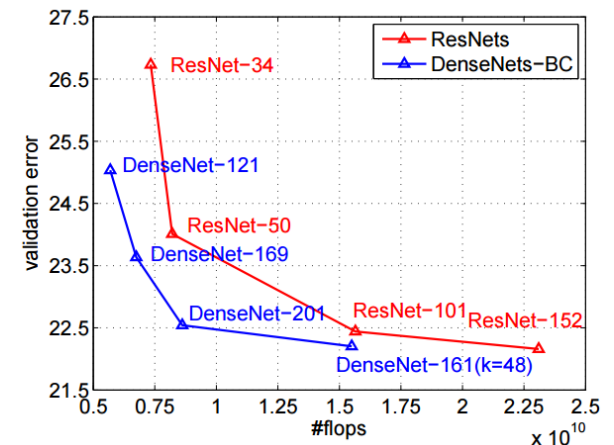
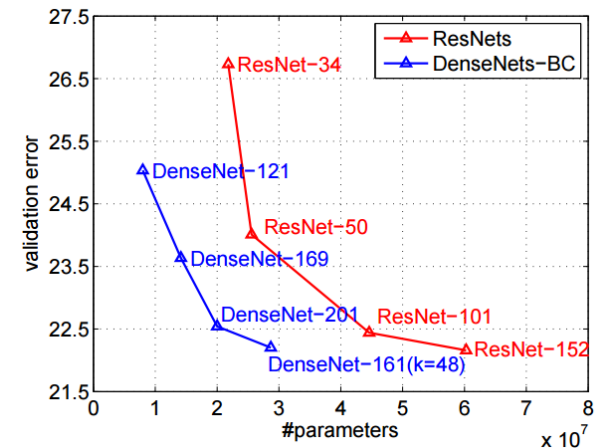
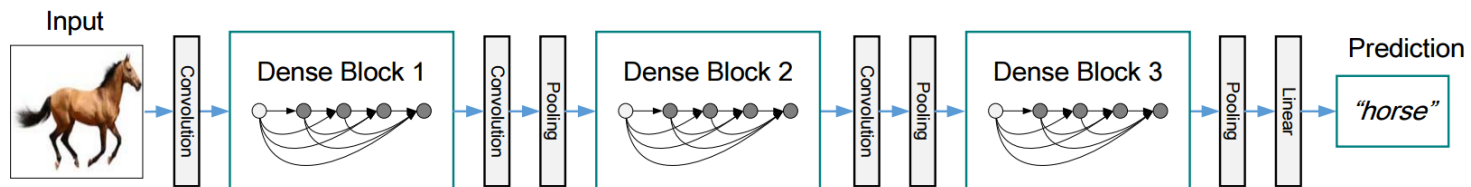
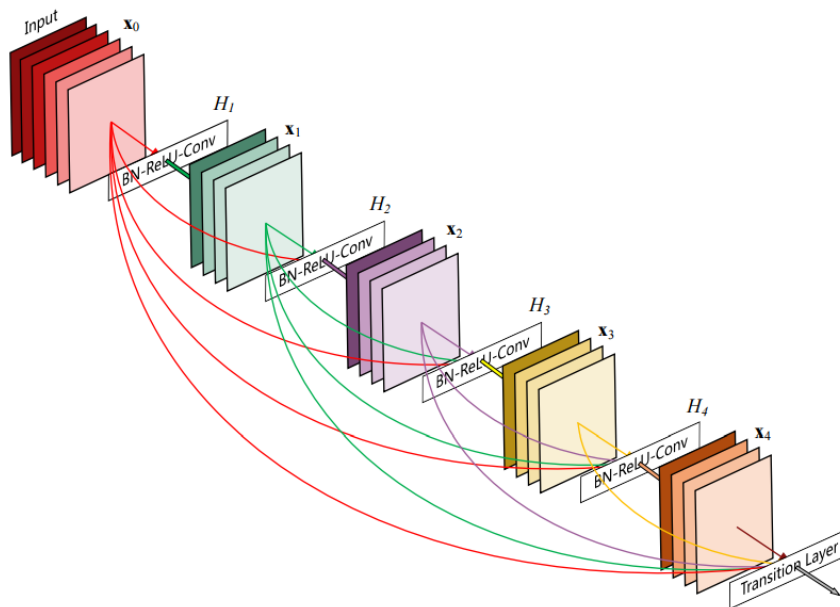
- How can we train very deep network?
  - Residual learning



| method                     | top-5 err. (test) |
|----------------------------|-------------------|
| VGG [41] (ILSVRC'14)       | 7.32              |
| GoogLeNet [44] (ILSVRC'14) | 6.66              |
| VGG [41] (v5)              | 6.8               |
| PRReLU-net [13]            | 4.94              |
| BN-inception [16]          | 4.82              |
| <b>ResNet (ILSVRC'15)</b>  | <b>3.57</b>       |

# DenseNet [CVPR'17]

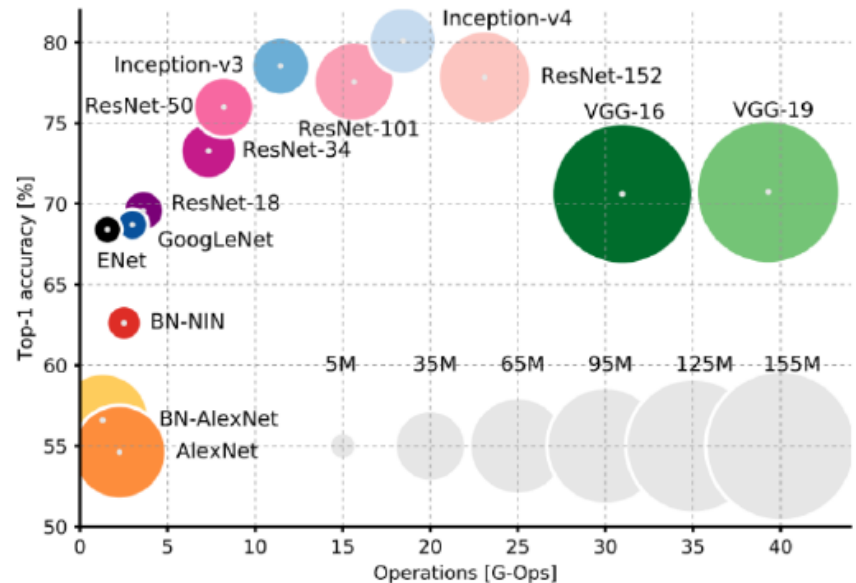
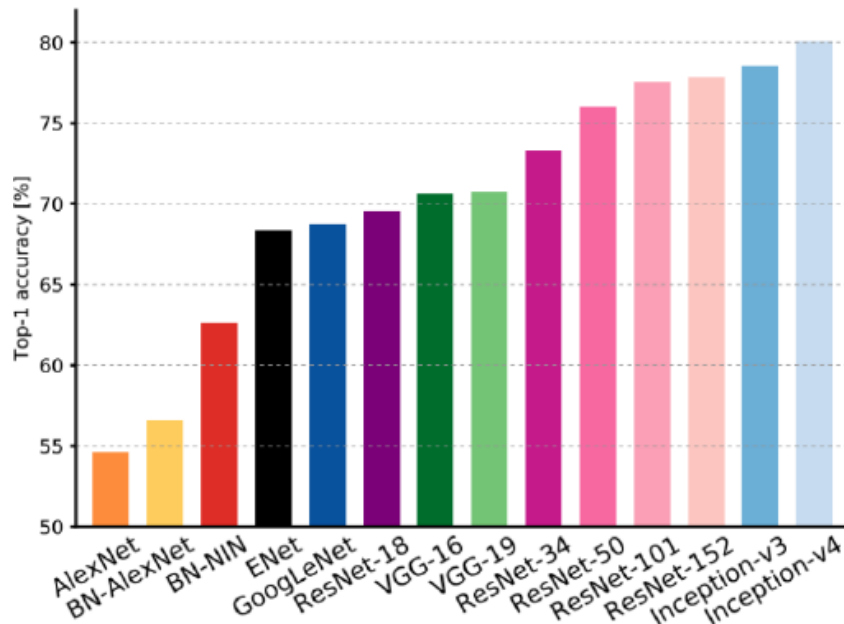
- Shorter connections (like ResNet) help
- Why not just connect them all?





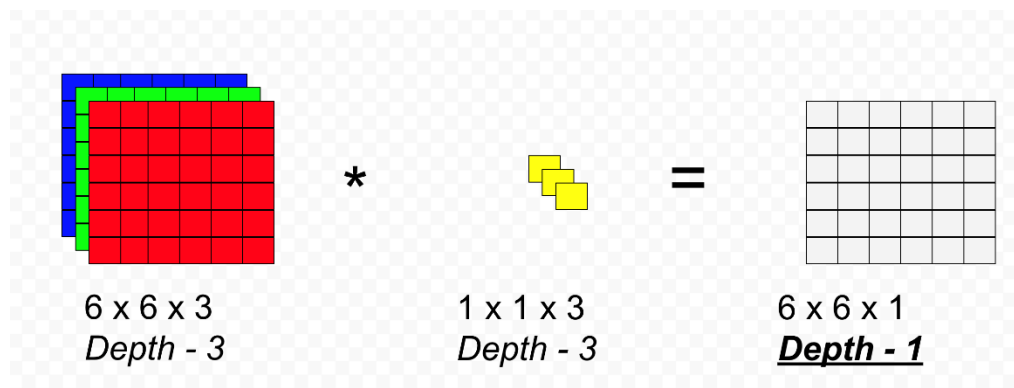
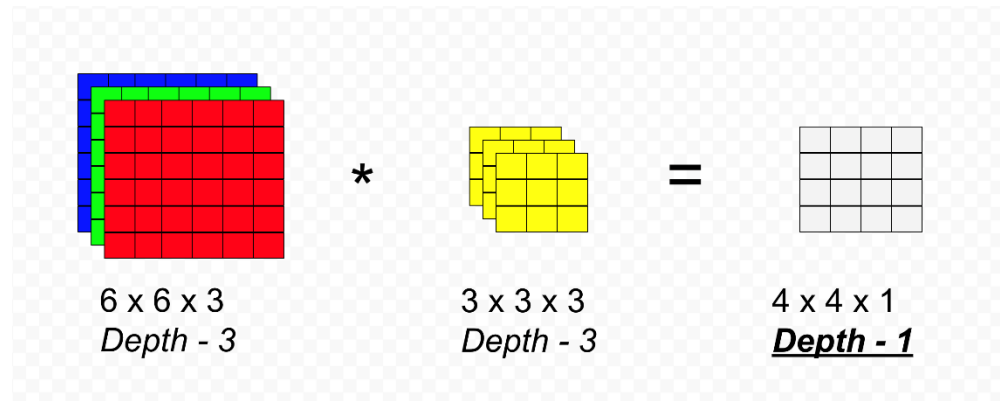
# Comparing Complexity

- ✓ Highest memory, most ops:
- ✓ Very efficient with moderate acc:
- ✓ Few ops but lots of parameters:
- ✓ Simple design, moderate efficiency yet high accuracy:



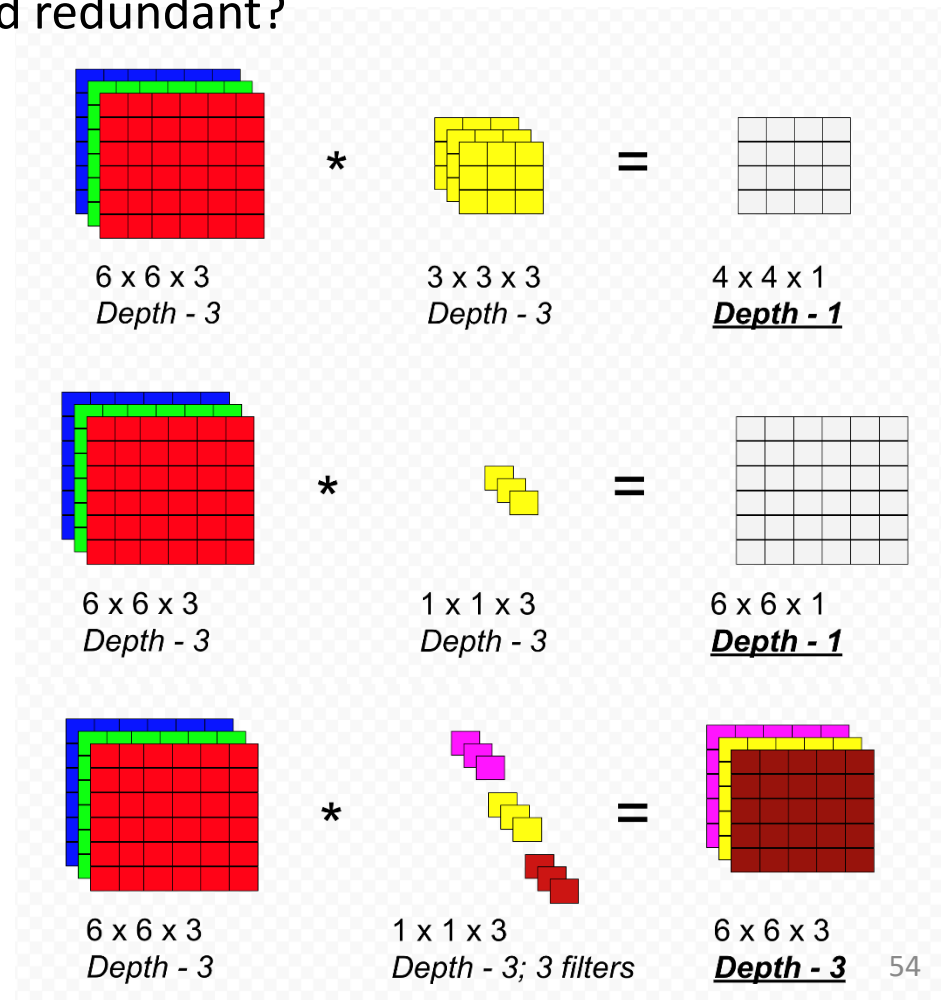
# Btw, what is 1x1 Convolution?

- Doesn't 1x1 convolution sound redundant?
- Actually, it's for accelerating computation purposes



# What is 1x1 Convolution? (cont'd)

- Doesn't 1x1 convolution sound redundant?
- Simply speaking, it provides...
  - Dimension reduction
  - Additional nonlinearity



# What is 1x1 Convolution? (cont'd)

- **Example 1**

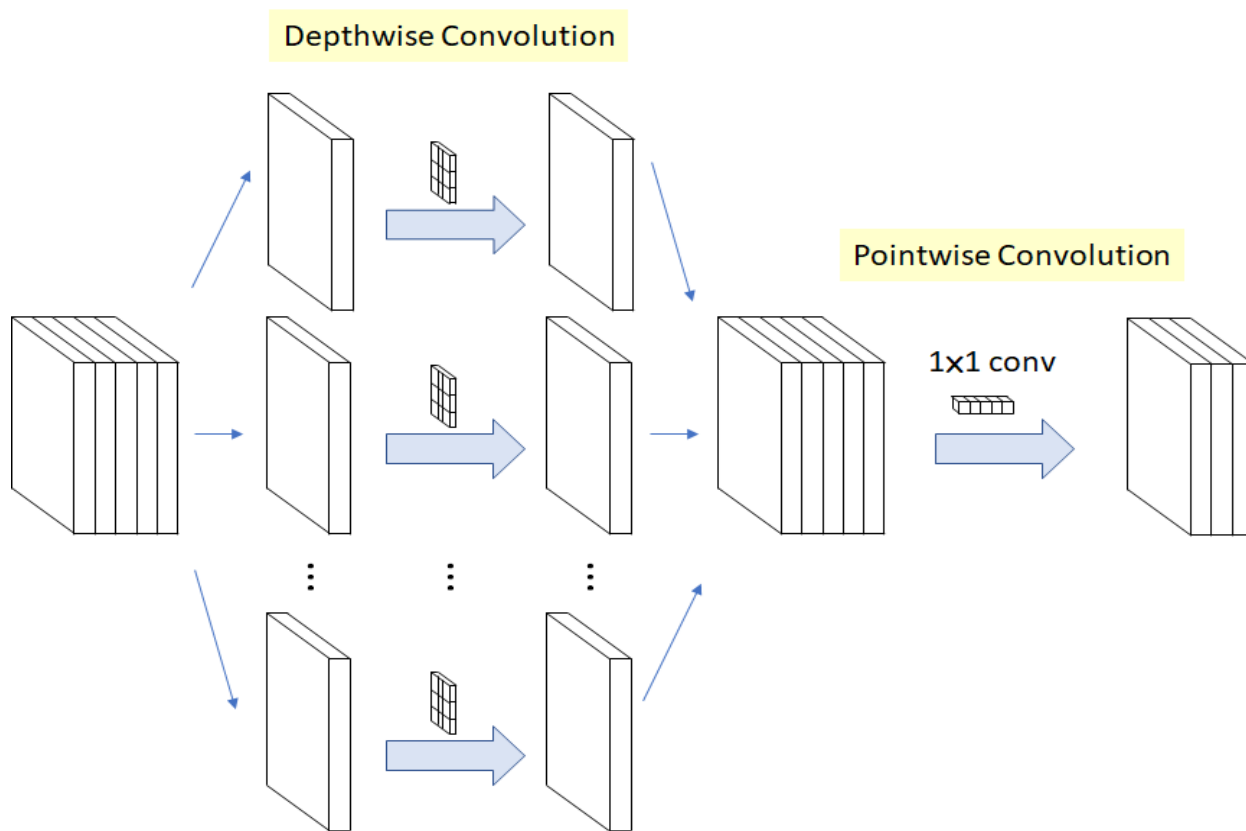
- **{28 x 28 x 192}** convolved with 32 {5 x 5x 192} kernels into **{28 x 28 x 32}**
- (5 x 5 x 192) muls x (28 x 28) pixels x 32 kernels ~ 120M muls

- **Example 2**

- **{28 x 28 x 192}** convolved with 16 {1 x 1x 192} kernels into {28 x 28 x 16}, followed by convolution with into 32 {5 x 5 x 16} kernels into **{28 x 28 x 32}**
- 192 mul x (28 x 28) pixels x 16 kernels ~ 2.4M
- (5 x 5 x 16) muls x (28 x 28) pixels x 32 kernels ~ 10M
- 12.4M (2.4M + 10M) << 120M; what's the price to pay?

# MobileNets: Tiny Networks for End Devices

- MobileNet V1
  - Depthwise & pointwise convolution



# MobileNets (cont'd)

- MobileNet V1

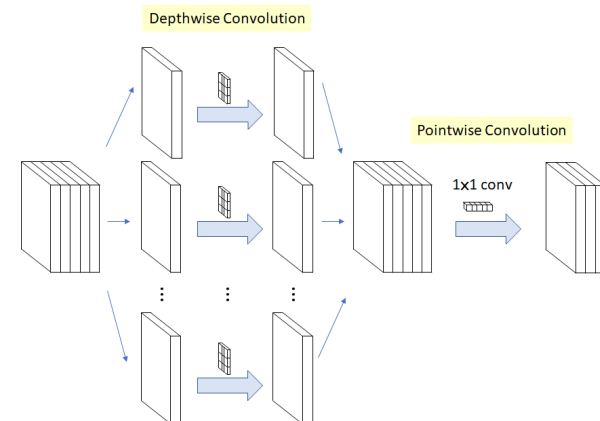
- Depthwise & pointwise convolution
- Reduced Computation

- Input feature map  $D_F \times D_F$  pixels with  $M$  channels, kernel size  $D_K$ , & output with  $N$  channels
- The ratio of required computation of depth+pointwise conv. and standard conv. is :

$$\begin{aligned}
 & \frac{\text{Depthwise Convolution} + \text{Pointwise Convolution}}{\text{Standard Convolution}} \\
 &= \frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} \\
 &= \frac{1}{N} + \frac{1}{D_K^2}
 \end{aligned}$$

- Thus, depth+pointwise convolution requires only  $1/N + 1/D_K^2$  of the computation cost compared with that of standard convolution.

- Variants of MobileNets are available!

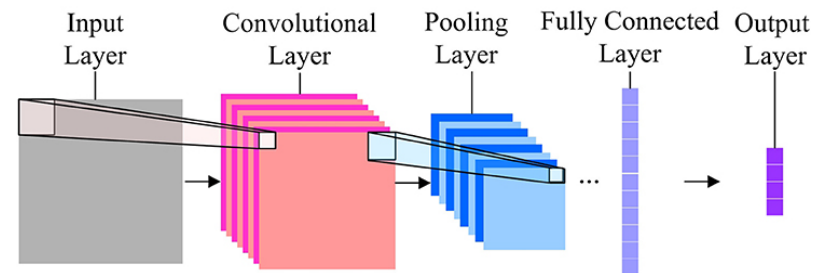
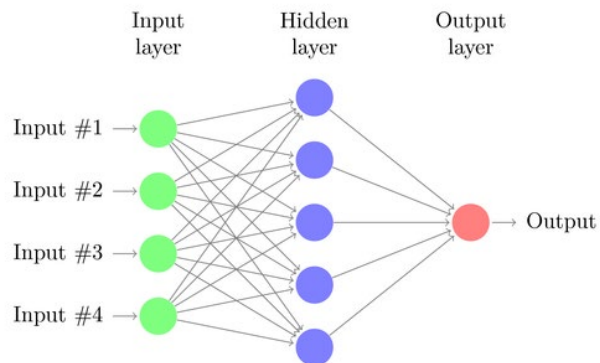
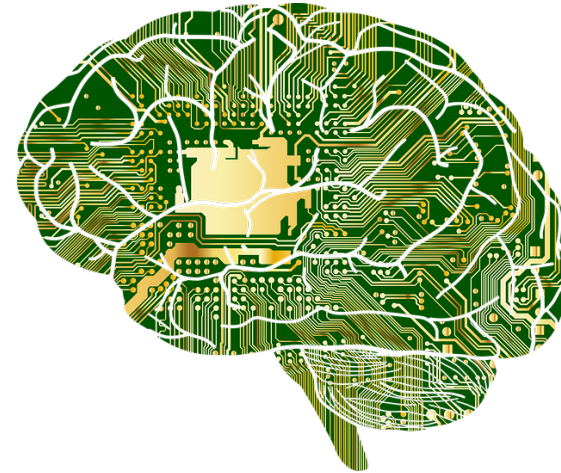


# Remarks

- CNN:
  - Reduce the number of parameters
  - Reduce the memory requirements
  - Make computation independent of the size of the image
- Neuroscience provides strong inspiration on the NN design, but little guidance on how to train CNNs.
- Few structures discussed: convolution, nonlinearity, pooling

# What's to Be Covered Today...

- Convolution Neural Networks (CNN)
  - Design of CNN
  - Variants of CNNs
  - Training Techniques for CNN





# Selected Tricks for Training Deep Learning Models

- ~~Backpropagation + stochastic gradient descent with momentum~~
  - [Neural Networks: Tricks of the Trade](#)
- Dropout
- Data augmentation
- Batch normalization

## Backpropagation: Simple Example

$$f(x, y, z) = (x + y) \cdot z$$

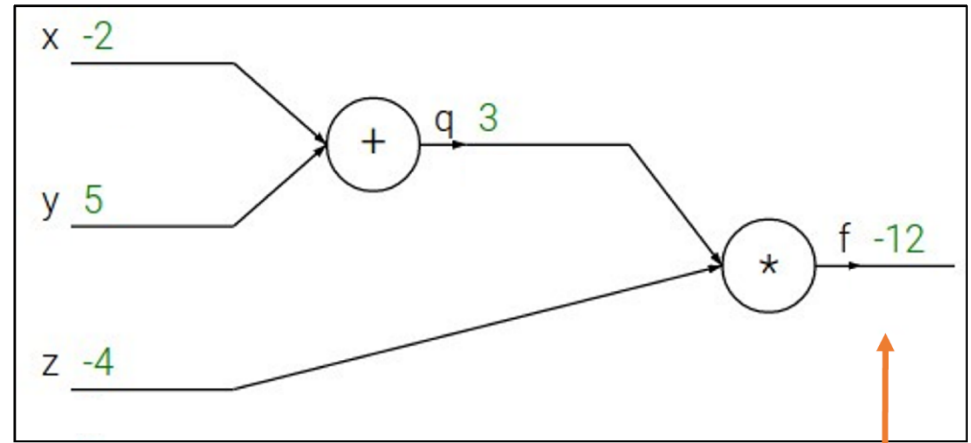
e.g.  $x = -2, y = 5, z = -4$

**1. Forward pass:** Compute outputs

$$q = x + y \quad f = q \cdot z$$

**2. Backward pass:** Compute derivatives

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

## Backpropagation: Simple Example

$$f(x, y, z) = (x + y) \cdot z$$

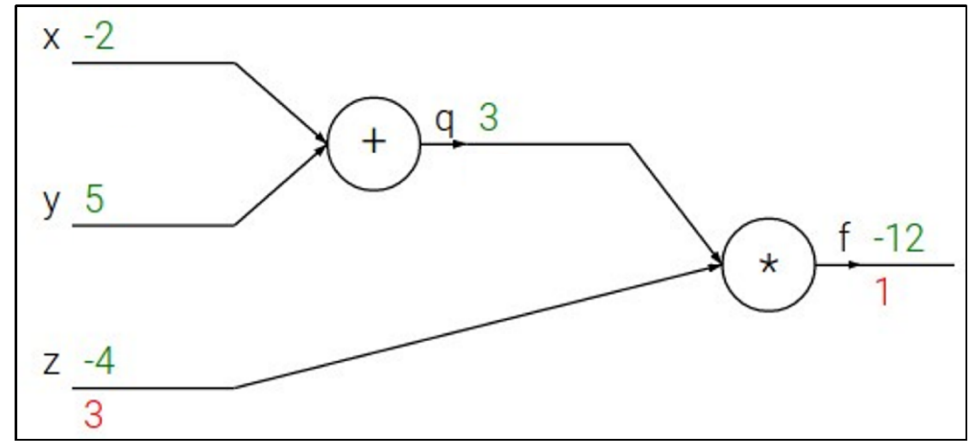
e.g.  $x = -2, y = 5, z = -4$

**1. Forward pass:** Compute outputs

$$q = x + y \quad \boxed{f = q \cdot z}$$

**2. Backward pass:** Compute derivatives

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\boxed{\frac{\partial f}{\partial z} = q}$$

## Backpropagation: Simple Example

$$f(x, y, z) = (x + y) \cdot z$$

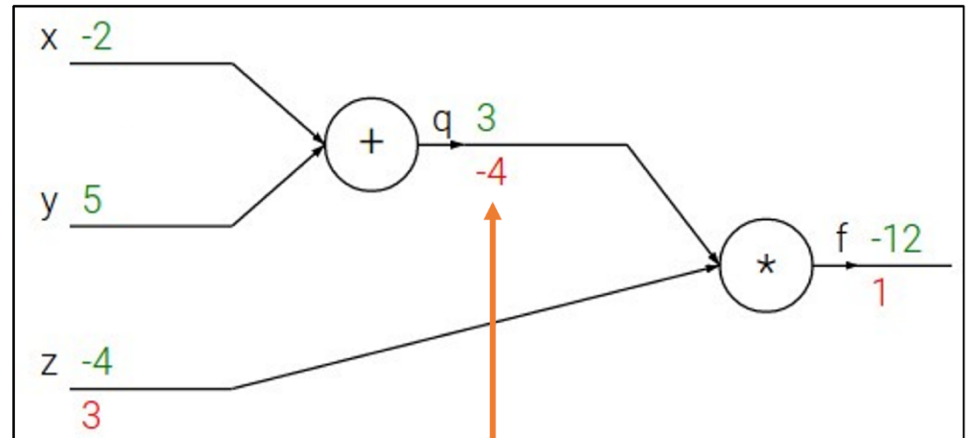
e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

**1. Forward pass:** Compute outputs

$$q = x + y \quad \boxed{f = q \cdot z}$$

**2. Backward pass:** Compute derivatives

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\boxed{\frac{\partial f}{\partial q} = z}$$

# Backpropagation: Simple Example

$$f(x, y, z) = (x + y) \cdot z$$

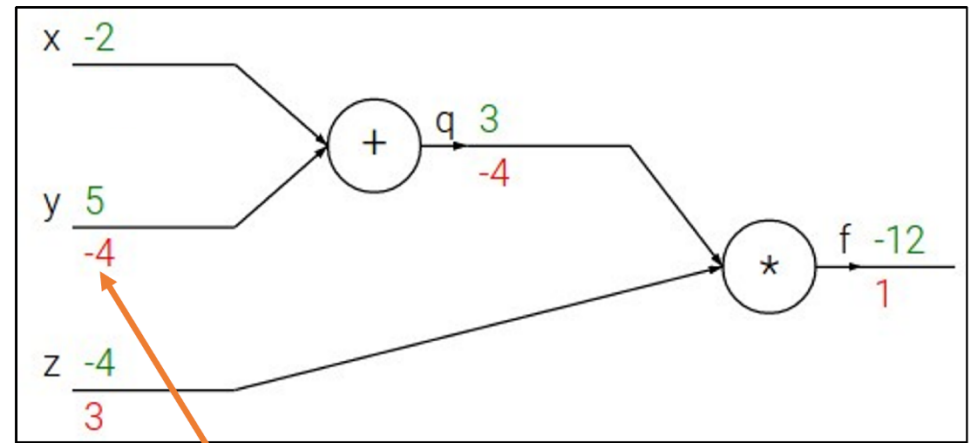
e.g.  $x = -2, y = 5, z = -4$

**1. Forward pass:** Compute outputs

$$q = x + y \quad f = q \cdot z$$

**2. Backward pass:** Compute derivatives

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain Rule

$$\frac{\partial f}{\partial y} = \frac{\partial q}{\partial y} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial y} = 1$$

Downstream  
Gradient

Local  
Gradient

Upstream  
Gradient

## Backpropagation: Simple Example

$$f(x, y, z) = (x + y) \cdot z$$

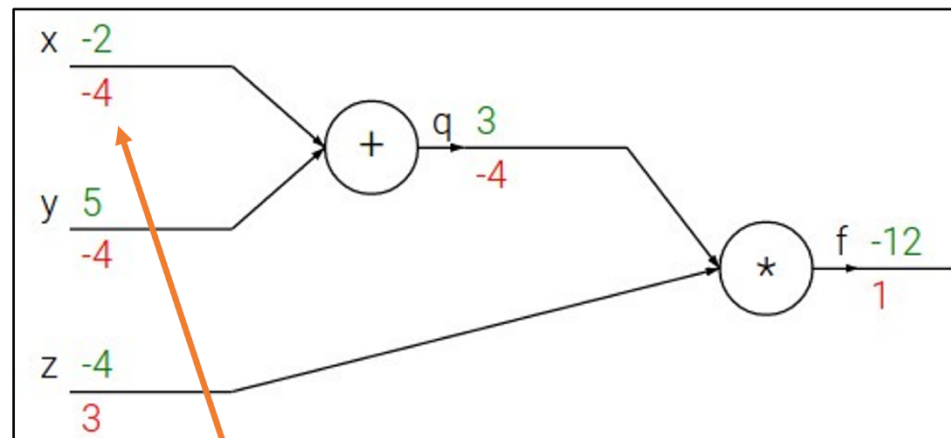
e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$

**1. Forward pass:** Compute outputs

$$q = x + y \quad f = q \cdot z$$

**2. Backward pass:** Compute derivatives

Want:  $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

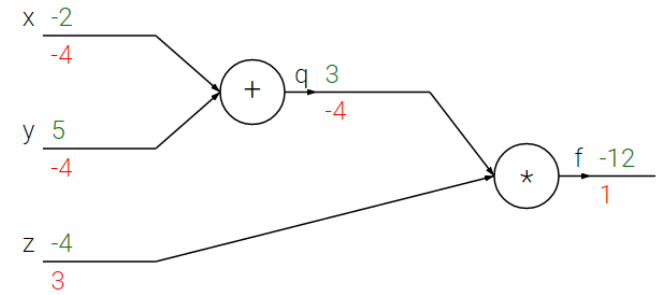
$$\frac{\partial q}{\partial x} = 1$$

Downstream  
Gradient

Local  
Gradient

Upstream  
Gradient

$$f(x, y, z) = (x + y)z = qz$$



$$q = x + y$$

$$\frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz$$

$$\frac{\partial f}{\partial q} = z, \quad \frac{\partial f}{\partial z} = q$$

**Chain rule:**

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

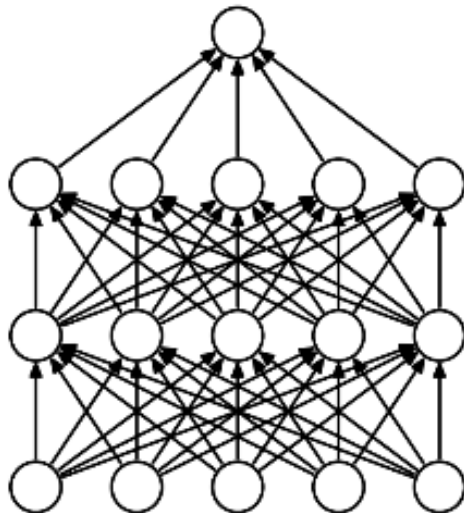
```

# set some inputs
x = -2; y = 5; z = -4

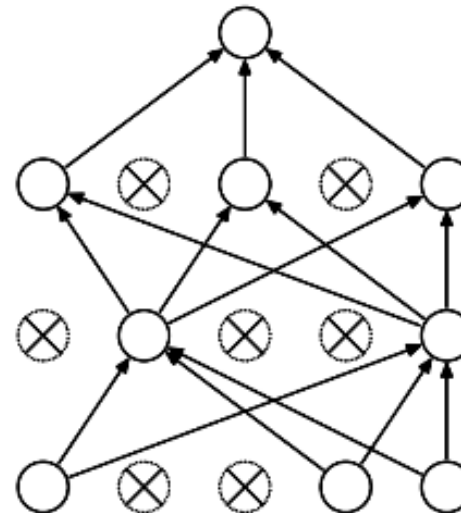
# perform the forward pass
q = x + y # q becomes 3
f = q * z # f becomes -12

# perform the backward pass (backpropagation) in reverse order:
# first backprop through f = q * z
dfd_z = q # df/dz = q, so gradient on z becomes 3
dfd_q = z # df/dq = z, so gradient on q becomes -4
# now backprop through q = x + y
dfd_x = 1.0 * dfdq # dq/dx = 1. And the multiplication here is the chain rule!
dfd_y = 1.0 * dfdq # dq/dy = 1
  
```

# Dropout



(a) Standard Neural Net



(b) After applying dropout.

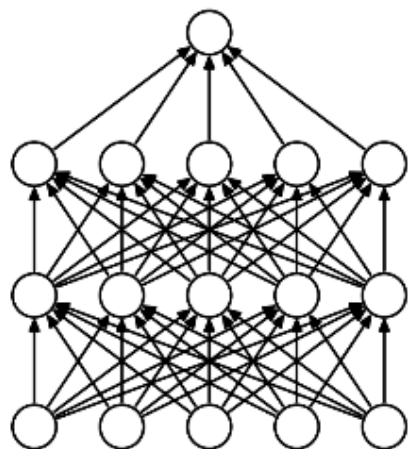
Intuition: successful **conspiracies**

Example: 50 people planning a conspiracy

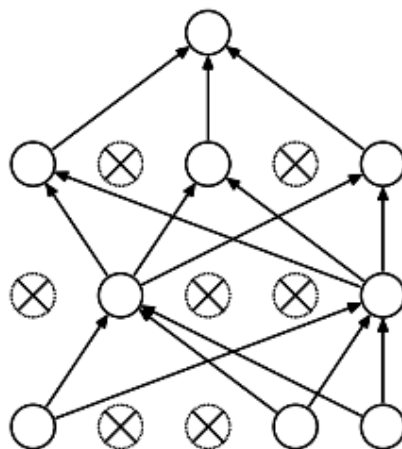
- Strategy A: plan a big conspiracy involving 50 people
  - Likely to fail. 50 people need to play their parts correctly.
- Strategy B: plan 10 conspiracies each involving 5 people
  - Likely to succeed!



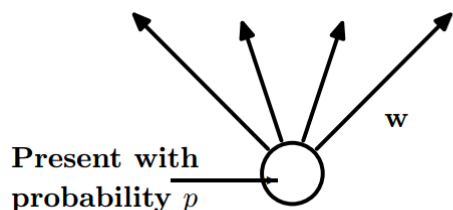
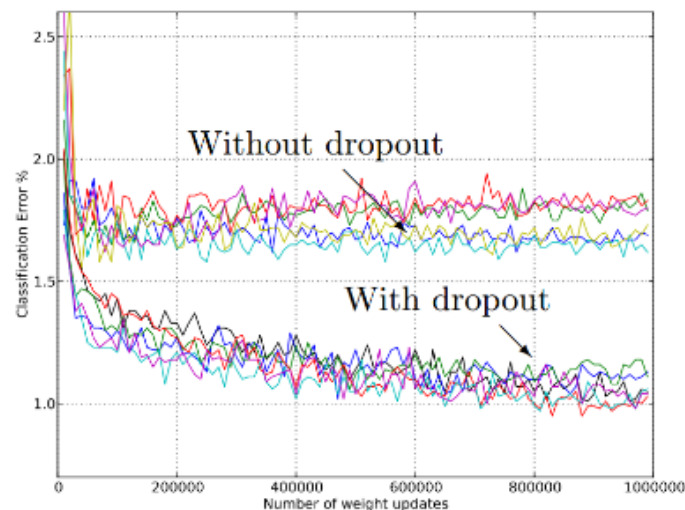
# Dropout



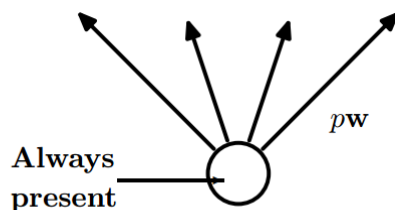
(a) Standard Neural Net



(b) After applying dropout.



(a) At training time



(b) At test time

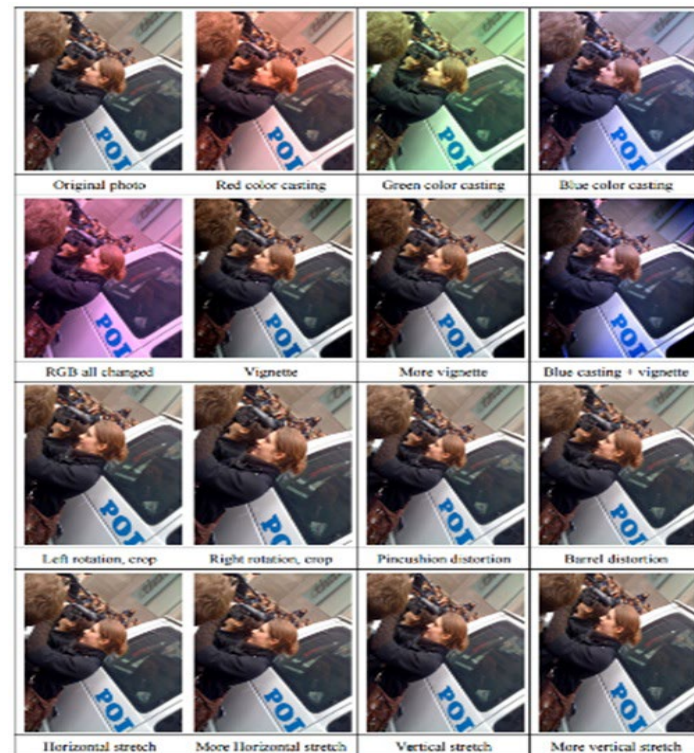
**Main Idea:** approximately combining exponentially many different neural network architectures efficiently

| Model                                                    | Top-1 (val) | Top-5 (val) | Top-5 (test) |
|----------------------------------------------------------|-------------|-------------|--------------|
| SVM on Fisher Vectors of Dense SIFT and Color Statistics | -           | -           | 27.3         |
| Avg of classifiers over FVs of SIFT, LBP, GIST and CSIFT | -           | -           | 26.2         |
| Conv Net + dropout (Krizhevsky et al., 2012)             | 40.7        | 18.2        | -            |
| Avg of 5 Conv Nets + dropout (Krizhevsky et al., 2012)   | 38.1        | 16.4        | 16.4         |

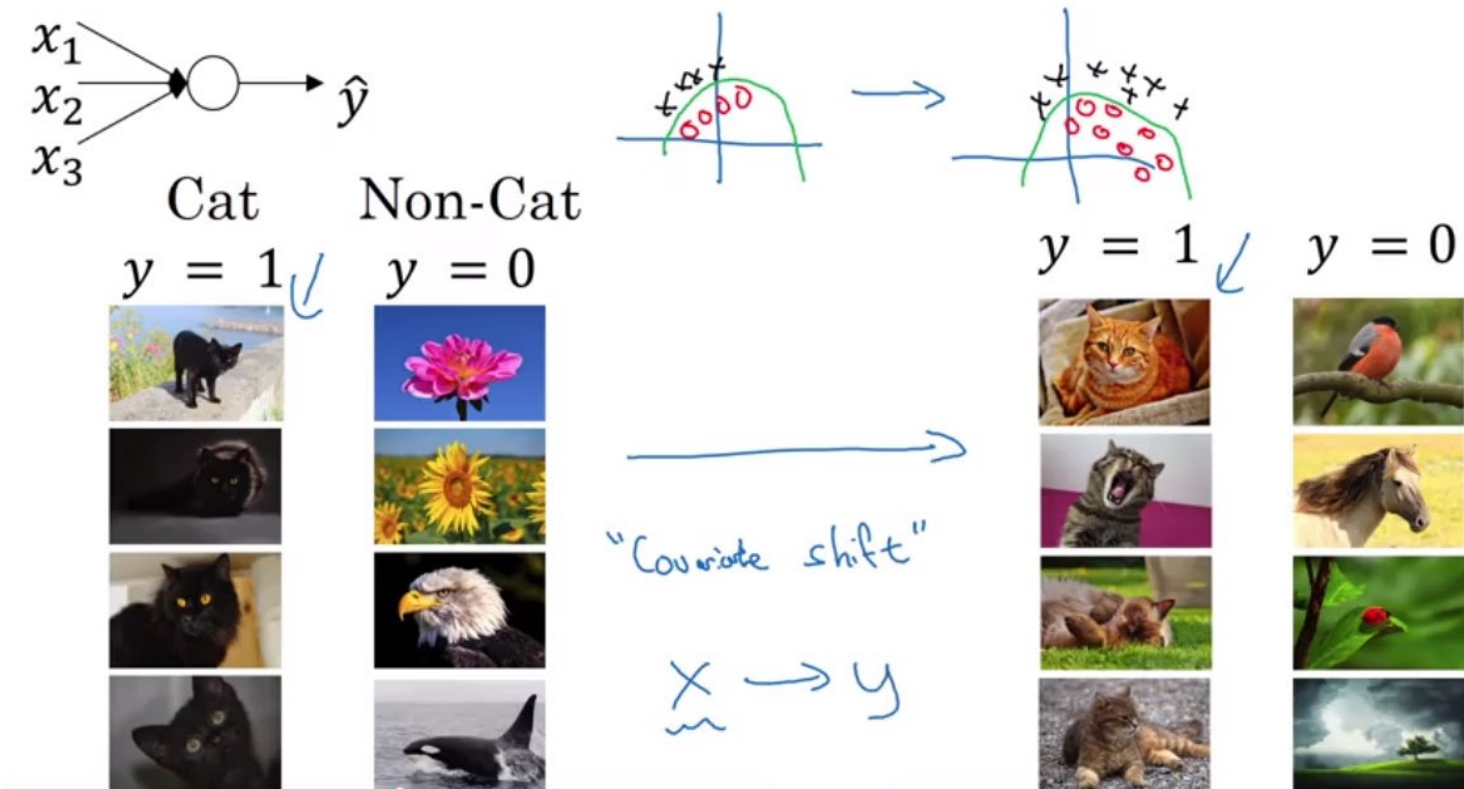
Table 6: Results on the ILSVRC-2012 validation/test set.

# Data Augmentation (Jittering)

- DL typically requires larger # of data for training
- Collecting data is time and cost consuming...
- Create *virtual* training samples
  - Horizontal flip
  - Random crop
  - Color casting
  - Geometric distortion and so on...
  - See any concerns?



# Batch Normalization



# Batch Normalization

**Input:** Values of  $x$  over a mini-batch:  $\mathcal{B} = \{x_1 \dots x_m\}$ ;  
Parameters to be learned:  $\gamma, \beta$

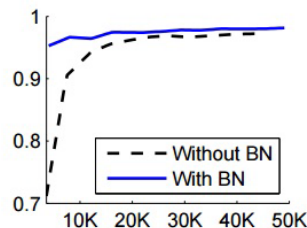
**Output:**  $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

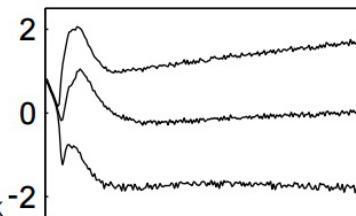
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

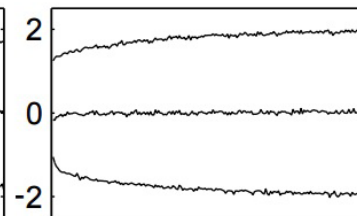
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$



(a)



(b) Without BN



(c) With BN

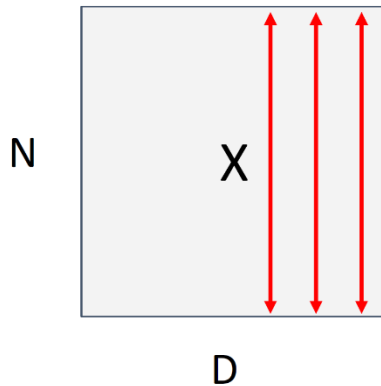
# Batch Normalization (cont'd)

- Remarks
  - Differentiable function; back propagation OK

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

- Procedure

**Input:**  $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean  
across N samples

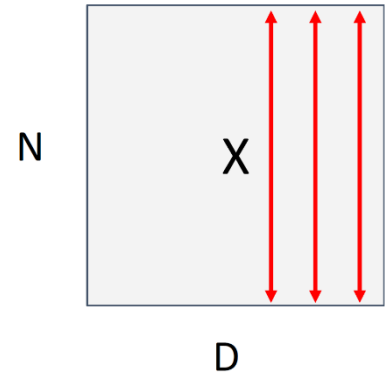
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel std  
across N samples

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,  
Shape is N x D

# Batch Normalization (cont'd)



- Remarks

- Differentiable function; back propagation OK

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

- Procedure (cont'd)

- With learnable scale and shift parameters  $\gamma$  and  $\beta$  to alleviate the hard constraint of zero-mean and unit variance

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized  $x$ ,  
Shape is  $N \times D$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,  
Shape is  $N \times D$

- Mean and variance estimated from each mini-batch during training
  - What about inference/testing?

$\mu_j =$  (Running) average of values seen during training      Per-channel mean across  $N$  samples

$\sigma_j^2 =$  (Running) average of values seen during training      Per-channel std across  $N$  samples

# Instance Normalization in CNN

**Batch Normalization** for  
convolutional networks

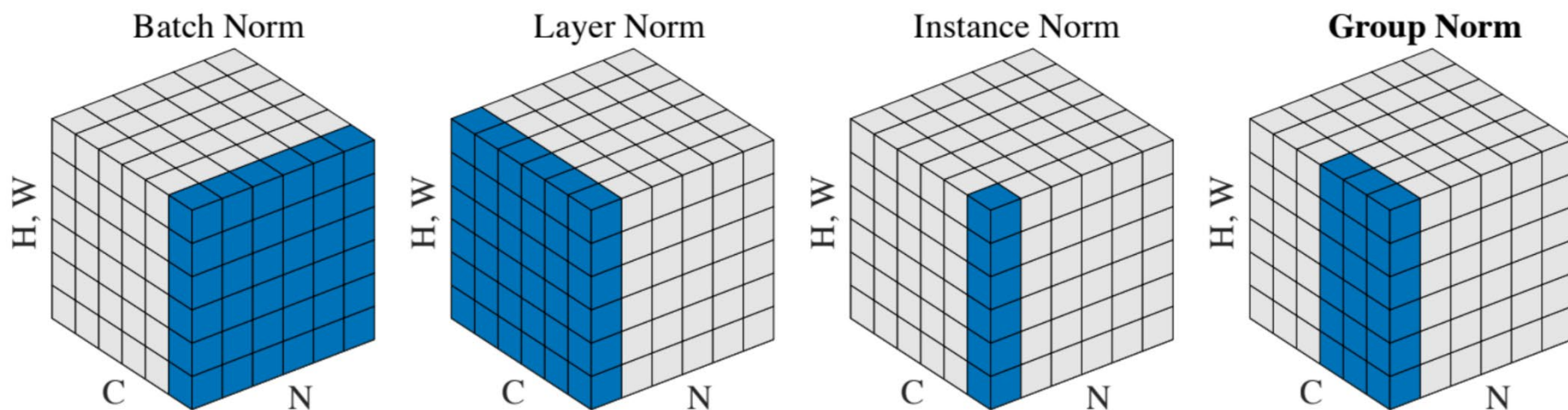
$$\begin{array}{l} \mathbf{x} : \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W} \\ \text{Normalize} \quad \downarrow \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma} : \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \boldsymbol{\gamma}, \boldsymbol{\beta} : \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

**Instance Normalization** for  
convolutional networks  
Same behavior at train / test!

$$\begin{array}{l} \mathbf{x} : \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W} \\ \text{Normalize} \quad \quad \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma} : \mathbf{N} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \boldsymbol{\gamma}, \boldsymbol{\beta} : \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$



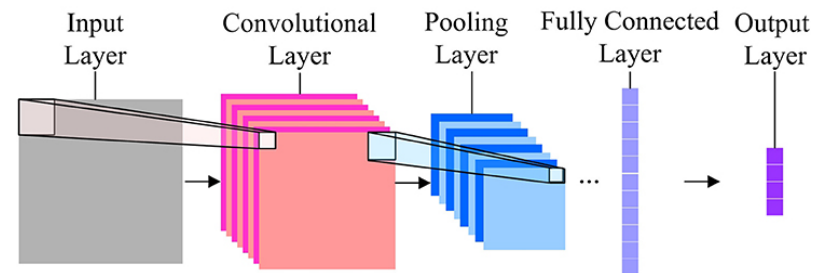
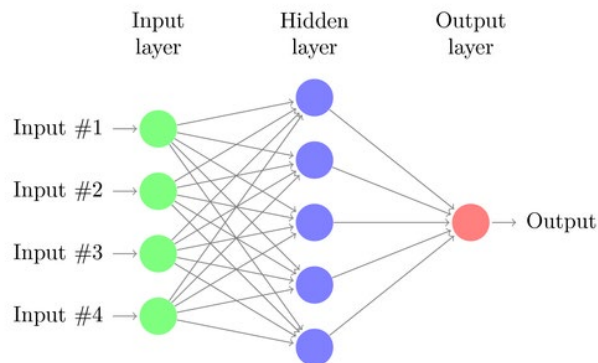
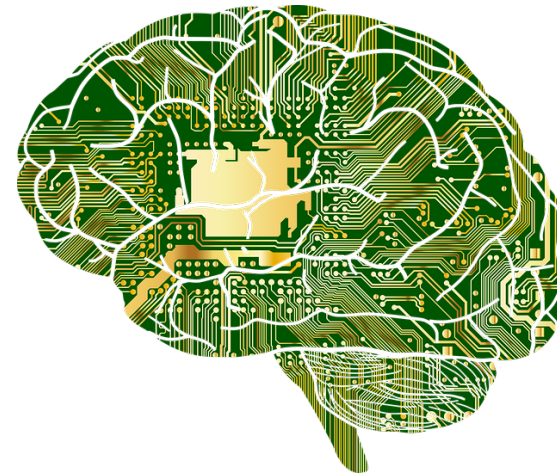
# Variants of Normalization in Training CNN





# What's to Be Covered Today...

- Convolution Neural Networks (CNN)
  - Design of CNN
  - Variants of CNNs
  - Training Techniques for CNN
  - Self-Supervised Learning for CNN



# Supervised Learning

- Most DL models are learned in a supervised fashion...

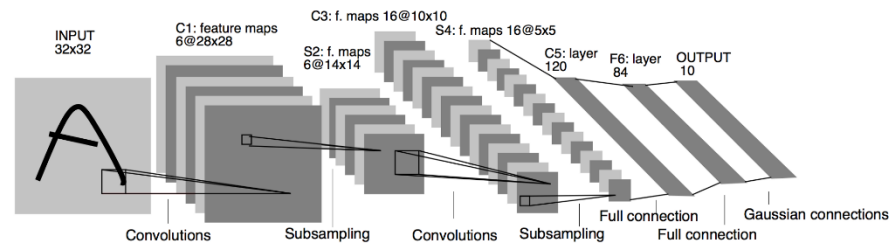
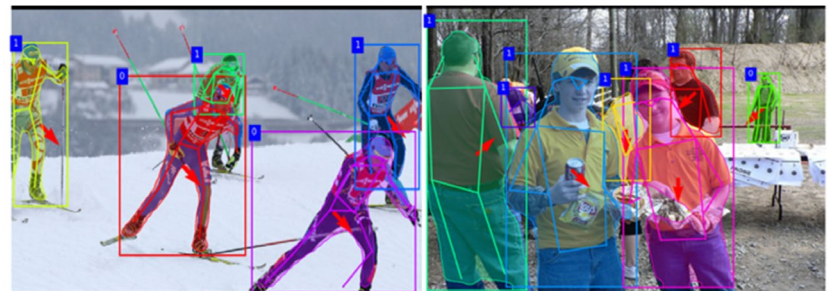


Image classification



Semantic  
segmentation



Object detection

- In real world scenarios, data-annotation is quite **time-consuming**
- Could one exploit supervised signals from **unlabeled** data?

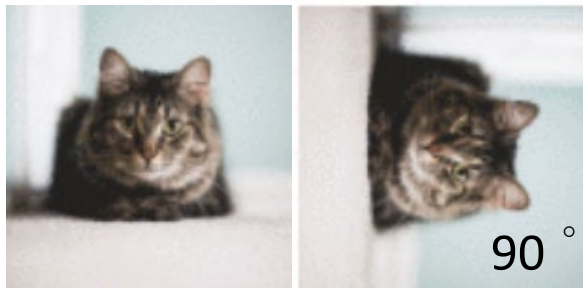


# Self-Supervised Learning (SSL)

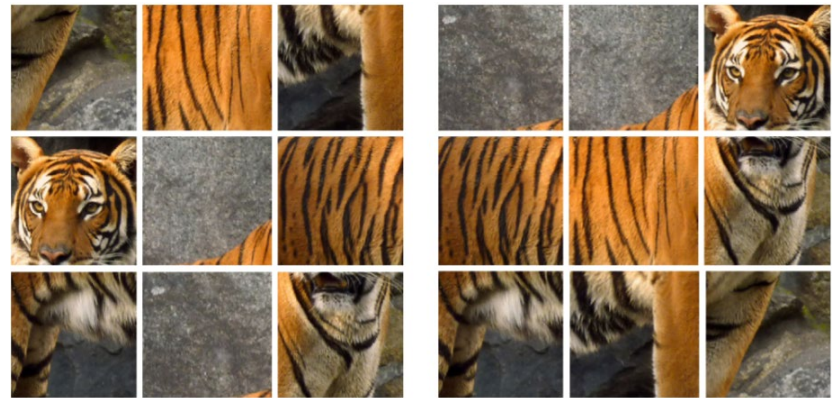
- Learning (somewhat) discriminative feature representations from **unlabeled** data
- Create self-supervised tasks via **data augmentation**



Colorization



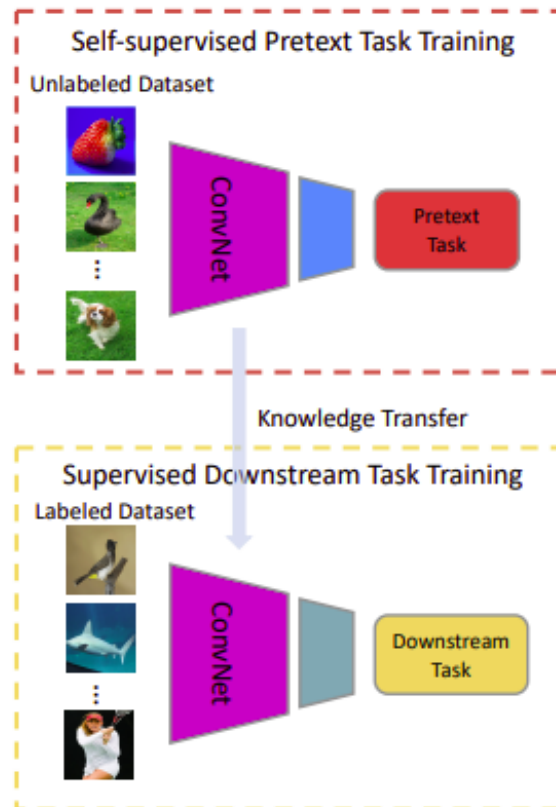
Rotation



Jigsaw Puzzle

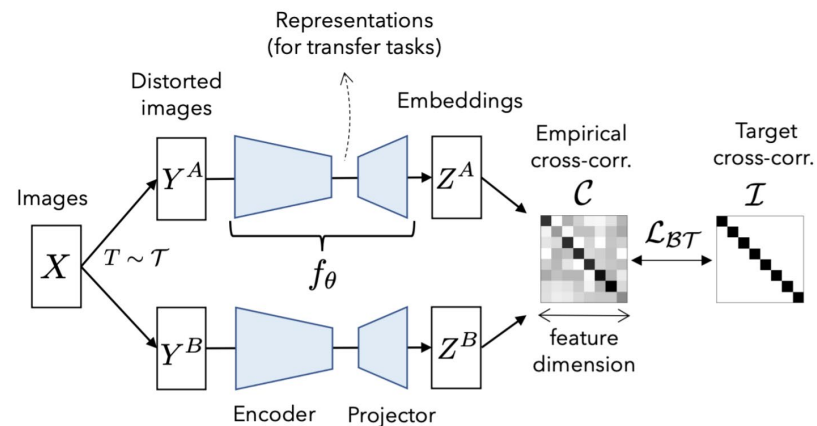
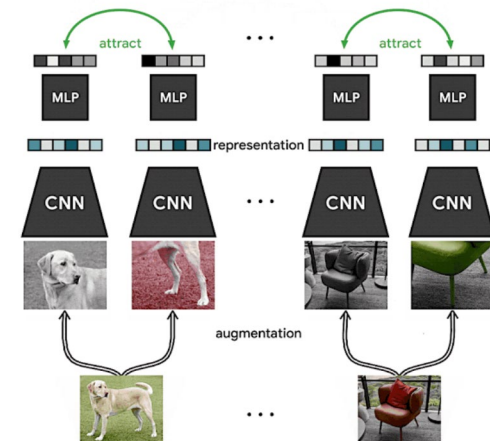
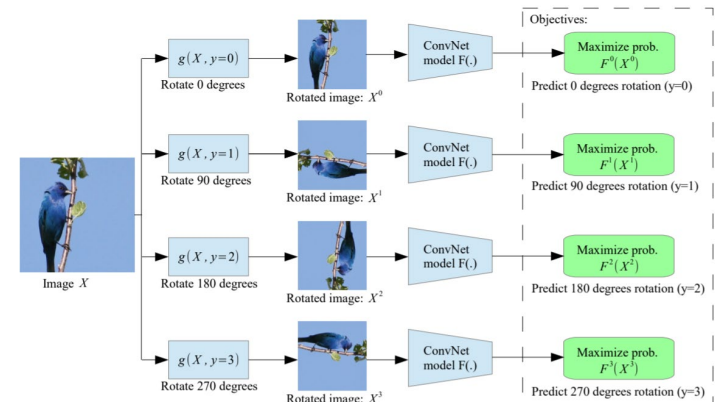
# A Typical SSL Procedure

- Stage 1: Self-Supervised Pretraining (w/ a *large* # of **unlabeled data**)
- Stage 2: Supervised Fine-tuning (w/ a *small* # of **labeled data**)
- Often performs favorably against models trained in a fully-supervised fashion



# Selected SSL Techniques

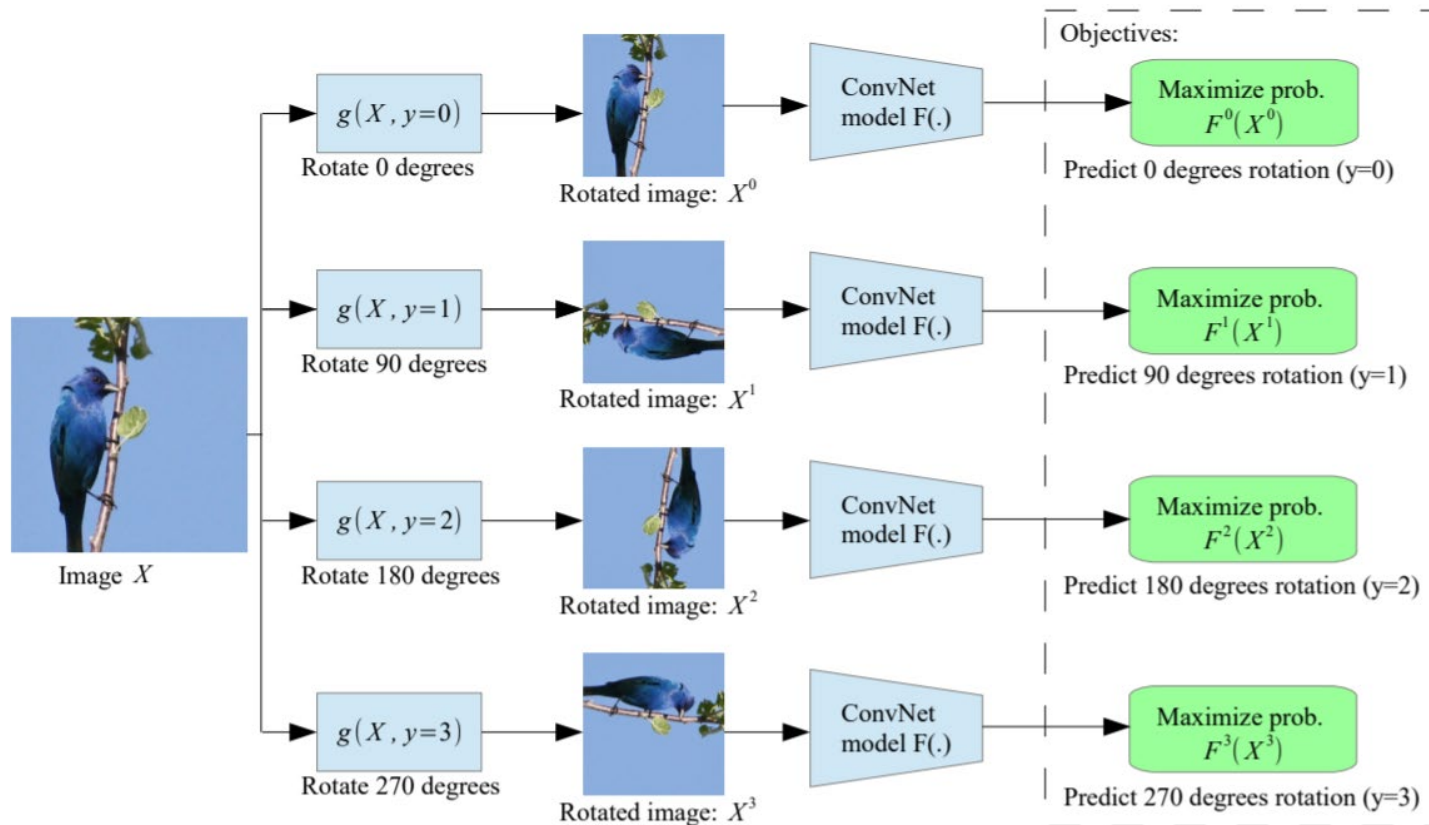
- Pretext Tasks
  - Jigsaw (ECCV'16)
  - RotNet (ICLR'18)
- Contrastive Learning
  - CPC (ICML'20)
  - SimCLR (ICML'20)
- Learning w/o negative samples
  - BYOL (NeurIPS'20)
  - Barlow Twins (ICML'21)





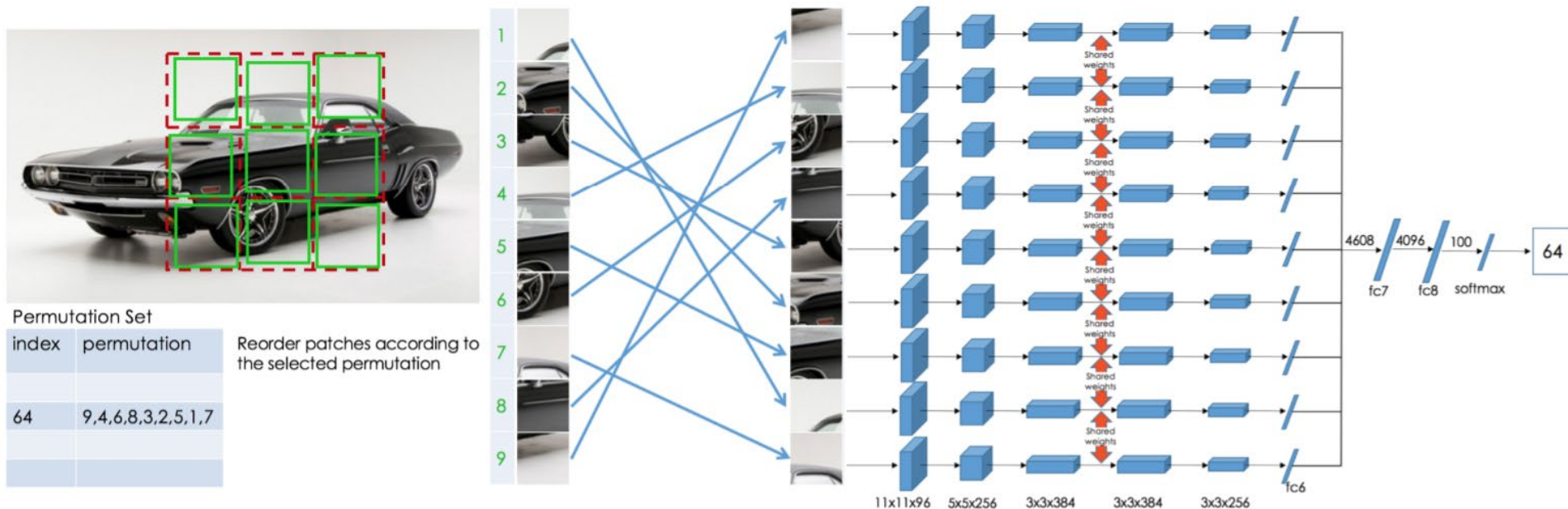
# RotNet

- Learning to predict the **rotation** angle



# Jigsaw Puzzle

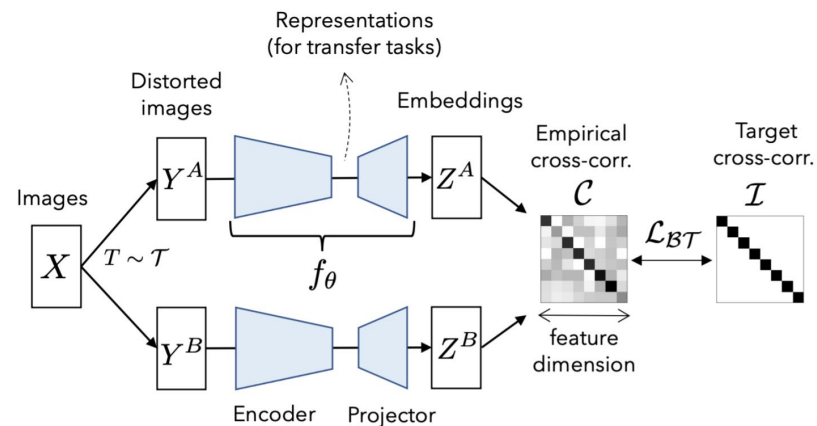
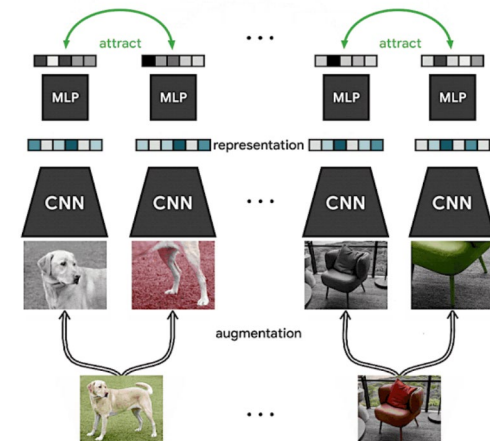
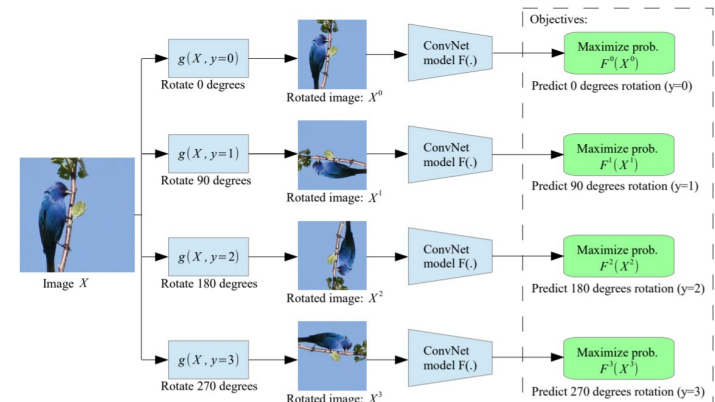
- Assign the **permutation index** and perform augmentation
- Solve jigsaw puzzle by predicting the permutation index





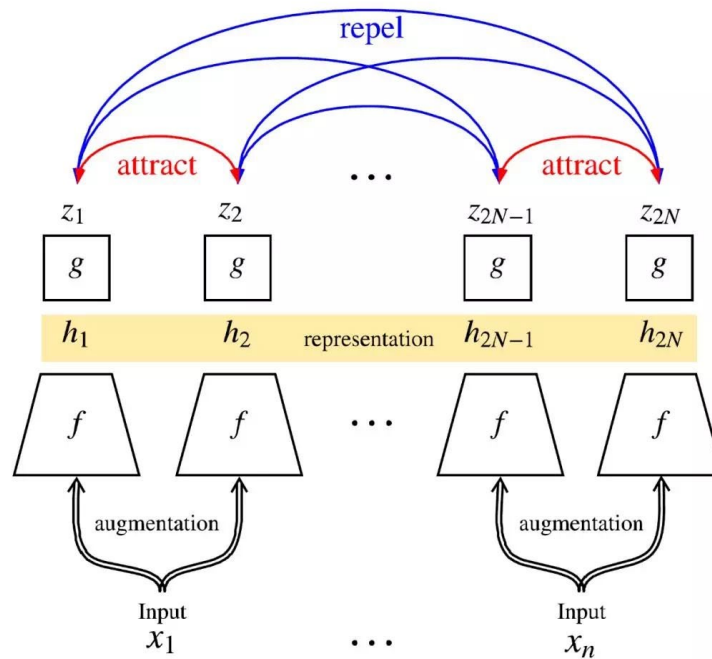
# Selected SSL Techniques

- Pretext Tasks
  - Jigsaw (ECCV'16)
  - RotNet (ICLR'18)
- Contrastive Learning
  - CPC (ICML'20)
  - SimCLR (ICML'20)
- Learning w/o negative samples
  - BYOL (NeurIPS'20)
  - Barlow Twins (ICML'21)



# SimCLR

- **Attract** augmented images and **repel** negative samples
- Once SSL training is complete, ignore **projection head**  $g$  & take **encoder**  $f$  for downstream tasks



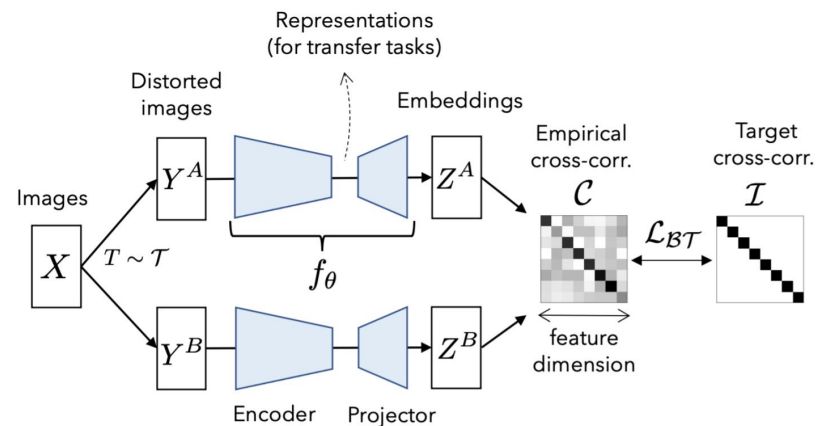
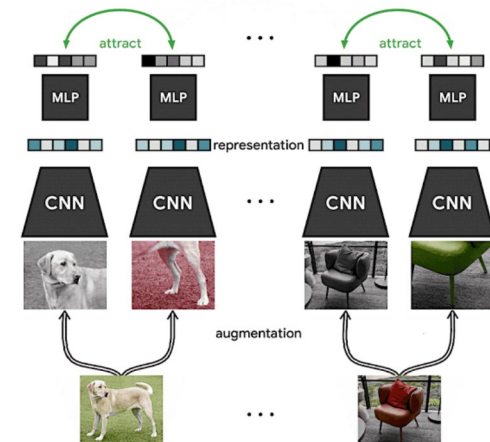
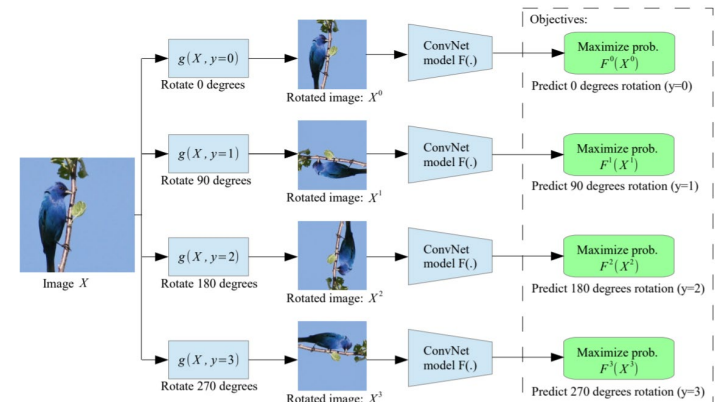
# SimCLR

- Experiments on semi-supervised settings

| Method                                             | Architecture   | Label fraction |             |
|----------------------------------------------------|----------------|----------------|-------------|
|                                                    |                | 1%             | 10%         |
|                                                    |                | Top 5          |             |
| Supervised baseline                                | ResNet-50      | 48.4           | 80.4        |
| <i>Methods using other label-propagation:</i>      |                |                |             |
| Pseudo-label                                       | ResNet-50      | 51.6           | 82.4        |
| VAT+Entropy Min.                                   | ResNet-50      | 47.0           | 83.4        |
| UDA (w. RandAug)                                   | ResNet-50      | -              | 88.5        |
| FixMatch (w. RandAug)                              | ResNet-50      | -              | 89.1        |
| S4L (Rot+VAT+En. M.)                               | ResNet-50 (4×) | -              | 91.2        |
| <i>Methods using representation learning only:</i> |                |                |             |
| InstDisc                                           | ResNet-50      | 39.2           | 77.4        |
| BigBiGAN                                           | RevNet-50 (4×) | 55.2           | 78.8        |
| PIRL                                               | ResNet-50      | 57.2           | 83.8        |
| CPC v2                                             | ResNet-161(*)  | 77.9           | 91.2        |
| SimCLR (ours)                                      | ResNet-50      | 75.5           | 87.8        |
| SimCLR (ours)                                      | ResNet-50 (2×) | 83.0           | 91.2        |
| SimCLR (ours)                                      | ResNet-50 (4×) | <b>85.8</b>    | <b>92.6</b> |

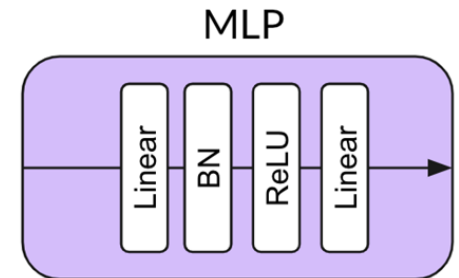
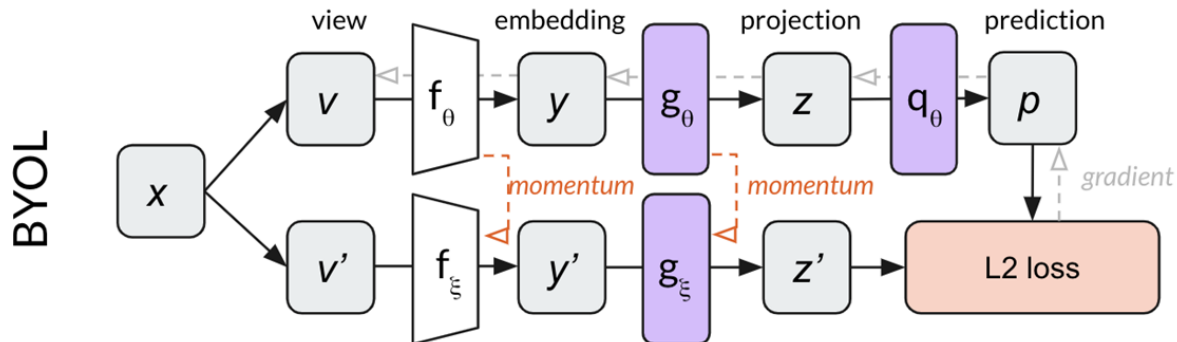
# Selected SSL Techniques

- Pretext Tasks
  - Jigsaw (ECCV'16)
  - RotNet (ICLR'18)
- Contrastive Learning
  - CPC (ICML'20)
  - SimCLR (ICML'20)
- Learning w/o negative samples
  - BYOL (NeurIPS'20)
  - Barlow Twins (ICML'21)



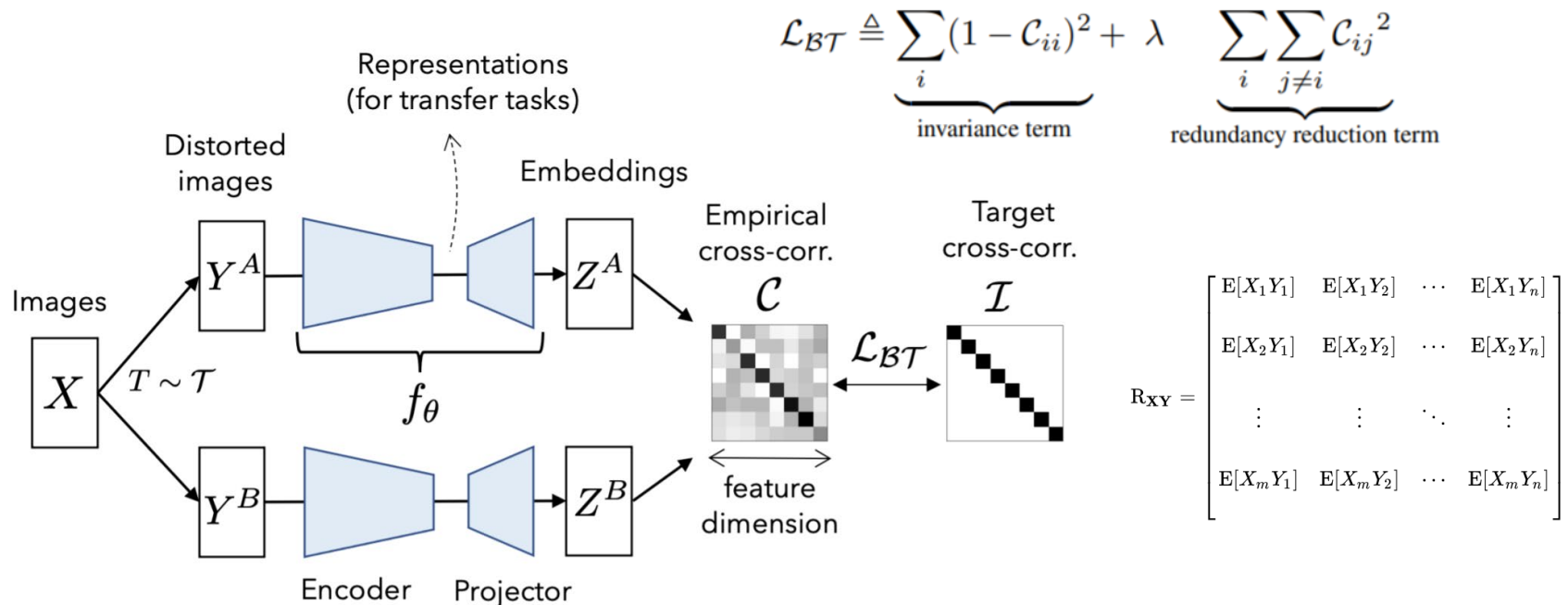
# BYOL (Bootstrap Your Own Latent)

- No need of negative pairs
- Introduce the **predictor** for architecture asymmetry to avoid model collapse
- Model update via Exponential Moving Average (**EMA**)



# Barlow Twins

- Still require assumption of positive/negative samples
- Make the output cross-correlation matrix close to identity matrix



# Barlow Twins

- Experiments on classification

| Method              | Top-1       |             | Top-5       |             |
|---------------------|-------------|-------------|-------------|-------------|
|                     | 1%          | 10%         | 1%          | 10%         |
| Supervised          | 25.4        | 56.4        | 48.4        | 80.4        |
| PIRL                | -           | -           | 57.2        | 83.8        |
| SIMCLR              | 48.3        | 65.6        | 75.5        | 87.8        |
| BYOL                | 53.2        | 68.8        | 78.4        | 89.0        |
| SwAV                | 53.9        | <b>70.2</b> | 78.5        | <b>89.9</b> |
| BARLOW TWINS (ours) | <b>55.0</b> | 69.7        | <b>79.2</b> | 89.3        |

# Barlow Twins

- Experiments on detection and segmentation

| Method    | VOC07+12 det |             |             | COCO det    |                |                | COCO instance seg |                |                |
|-----------|--------------|-------------|-------------|-------------|----------------|----------------|-------------------|----------------|----------------|
|           | $AP_{all}$   | $AP_{50}$   | $AP_{75}$   | $AP^{bb}$   | $AP_{50}^{bb}$ | $AP_{75}^{bb}$ | $AP^{mk}$         | $AP_{50}^{mk}$ | $AP_{75}^{mk}$ |
| Sup.      | 53.5         | 81.3        | 58.8        | 38.2        | 58.2           | 41.2           | 33.3              | 54.7           | 35.2           |
| MoCo-v2   | <b>57.4</b>  | 82.5        | <b>64.0</b> | <b>39.3</b> | 58.9           | <b>42.5</b>    | <b>34.4</b>       | 55.8           | 36.5           |
| SwAV      | 56.1         | <b>82.6</b> | 62.7        | 38.4        | 58.6           | 41.3           | 33.8              | 55.2           | 35.9           |
| SimSiam   | 57           | 82.4        | 63.7        | 39.2        | <b>59.3</b>    | 42.1           | <b>34.4</b>       | <b>56.0</b>    | <b>36.7</b>    |
| BT (ours) | 56.8         | <b>82.6</b> | 63.4        | 39.2        | 59.0           | <b>42.5</b>    | 34.3              | <b>56.0</b>    | 36.5           |



# SSL Beyond Image Data

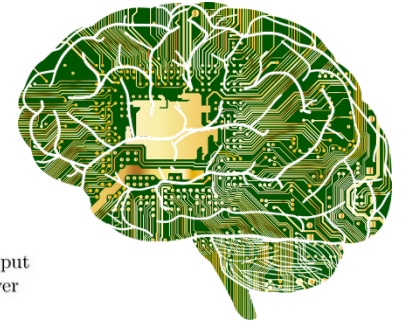
- What about videos?



J. Li et al., Learning to Learn from Noisy Labeled Data, *CVPR* 2019

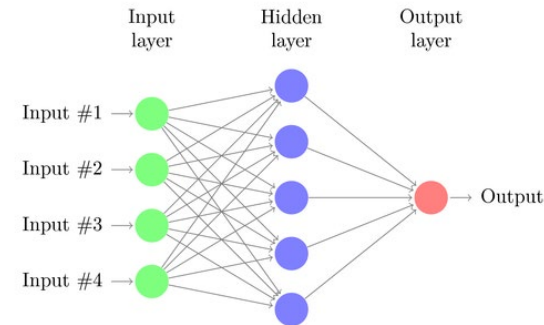
- You can come up with your own SSL strategy!

# What We've Covered Today...



- Convolution Neural Networks (CNN)

- Design of CNN
- Variants of CNNs
- Training Techniques for CNN
- Self-Supervised Learning for CNN



- I'm at KAIST next week; pre-recorded video lecture (image seg) for 9/17.
- HW #1 is out and due 9/24 23:59

